

R²GCurL: Reinforced Robust Knowledge Tracing via Dynamic Graph Curriculum Learning

TIANHAO PENG, Beihang University, Beijing, China and Nanyang Technological University, Singapore, Singapore

YANJUN PU, Beihang University, Beijing, China

YONGQI ZHANG, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China

YUCHEN LI, Baidu Inc., Beijing, China

JIAN REN, JIE LUO, and KUI ZHANG, Beihang University, Beijing, China

HAITAO YUAN, Tsinghua University, Beijing, China

SHUAIQIANG WANG and DAWEI YIN, Baidu Inc., Beijing, China

WENJUN WU, Beihang University, Beijing, China

With the rise of AI in education, knowledge tracing (KT) has become important for modeling students' knowledge from interaction data. However, existing methods still face three major challenges, including limited modeling of personalized exercise-concept relations, low robustness to noisy interactions, and inefficient training due to suboptimal data selection. To address these issues, we propose R²GCurL, a novel KT framework with two key designs. First, we recast KT as a graph classification problem and construct dynamic graphs from student responses, enabling the model to capture structural relations between exercises and concepts for more personalized KT. Second, we introduce a data-centric curriculum learning strategy based on dynamic graph entropy. Under our definition, pairwise dynamic graph entropy measures graph-transition continuity, where larger values indicate stronger structural similarity. Its sequence-level aggregation is used to derive a structure-aware difficulty signal for sample scheduling. On top of this, an RL-based scheduler further adapts batch selection based on model feedback and is especially beneficial under noisier and more unstable training regimes. Theoretical analysis shows that R²GCurL has lower computational complexity than existing graph-based KT models. Extensive experiments on five real-world datasets confirm its effectiveness, robustness, and generalizability, including as a plug-and-play enhancement for sequence-based KT models.

This work is supported in part by the National Natural Science Foundation of China under Grant Nos. 62441617 and 62276014. Y. Zhang's work is supported by Guangdong Provincial Natural Science Foundation 2025A1515010304, Guangdong Province Project 2024QN11X088, Guangzhou Science and Technology Planning Project 2025A03J4491.

The majority of this work was completed while Tianhao Peng was at Beihang University. He is currently affiliated with Nanyang Technological University.

Authors' Contact Information: Tianhao Peng, Beihang University, Beijing, China and Nanyang Technological University, Singapore, Singapore; e-mail: pengtianhao@buaa.edu.cn; Yanjun Pu, Beihang University, Beijing, China; e-mail: buaapyj@buaa.edu.cn; Yongqi Zhang, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China; e-mail: yongqizhang@hkust-gz.edu.cn; Yuchen Li, Baidu Inc, Shanghai, China; e-mail: yuchenli1230@gmail.com; Jian Ren, Beihang University, Beijing, China; e-mail: renjian@buaa.edu.cn; Jie Luo, Beihang University, Beijing, China; e-mail: luojie@buaa.edu.cn; Kui Zhang, Beihang University, Beijing, China; e-mail: zhangkui@buaa.edu.cn; Haitao Yuan, Tsinghua University, Beijing, China; e-mail: yhaitao45@163.com; Shuaiqiang Wang, Baidu Inc., Beijing, China; e-mail: shqiang.wang@gmail.com; Dawei Yin, Baidu Inc., Beijing, China; e-mail: yindawei@acm.org; Wenjun Wu (corresponding author), Beihang University, Beijing, China; e-mail: wwj09315@buaa.edu.cn.

*For correspondence, please contact Wenjun Wu, Jie Luo, and Haitao Yuan.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1558-2868/2026/7-ART129

<https://doi.org/10.1145/3818614>

CCS Concepts: • **Applied computing** → **Education**; • **Information systems** → **Data mining**; • **Computing methodologies** → **Neural networks**;

Additional Key Words and Phrases: Knowledge Tracing, Dynamic Graph, Curriculum Learning, Reinforcement Learning

ACM Reference format:

Tianhao Peng, Yanjun Pu, Yongqi Zhang, Yuchen Li, Jian Ren, Jie Luo, Kui Zhang, Haitao Yuan, Shuaiqiang Wang, Dawei Yin, and Wenjun Wu. 2026. R²GCuRL: Reinforced Robust Knowledge Tracing via Dynamic Graph Curriculum Learning. *ACM Trans. Inf. Syst.* 44, 6, Article 129 (July 2026), 32 pages.
<https://doi.org/10.1145/3818614>

1 Introduction

Intelligent Tutoring Systems (ITS) support personalized practice, feedback, and recommendation, and they now underpin many online learning platforms [3, 46, 48]. Within ITS, **Knowledge Tracing (KT)** estimates a learner’s evolving knowledge state from historical interactions and supports downstream tasks such as exercise recommendation, performance prediction, and personalized intervention [18, 23, 24, 27, 40, 58, 62]. As shown in Figure 1, each column represents an exercise at a particular timestamp, the colored circles in the concept rows indicate the concepts associated with that exercise, and the bottom row records whether the student answered correctly [9, 12, 48].

Traditional KT models, including Bayesian KT [36] and Item Response Theory-based methods [7, 29], are rooted in cognitive and educational theory, but they often make simplified assumptions about the learner state [46, 48]. As large-scale educational logs have become available, deep KT methods have emerged as the dominant way to model students’ evolving knowledge states from response sequences. Broadly, these methods fall into two groups: sequence-based and graph-based approaches. Sequence-based models use recurrent networks or self-attention to capture temporal dependencies [11, 35, 41, 43, 47, 52, 57, 61], but they usually encode exercise–concept structure only implicitly. Graph-based models instead represent structural dependencies among exercises and concepts explicitly [10, 46, 48], yet many rely on global or static structures that are not well suited to student-specific structural evolution over time. This gap motivates our focus on a method that preserves both temporal order and personalized exercise–concept structure. However, three data-related challenges remain central in current KT research.

- *Data structure limitations in modeling personalized concept relationships.* Most sequence-based KT models focus solely on modeling at the exercise or concept level, without explicitly capturing the relationships between exercises and concepts. In contrast, graph-based KT models are capable of representing such complex relationships, but they typically disregard student-specific interaction sequences and treat all learners as homogeneous. As a result, they fail to capture the dynamic interplay between (1) the structural dependencies among knowledge concepts and (2) individual learners’ evolving response histories. This limitation undermines the models’ ability to infer personalized knowledge states, ultimately reducing prediction accuracy in real-world educational scenarios.
- *Low robustness to noisy data.* Real-world online learning systems often encounter data quality issues, particularly inconsistent or misleading student interactions. Noisy interactions arise from various sources, including non-cognitive factors (e.g., disengagement) or suboptimal ITS recommendations that misalign exercise difficulty with a student’s proficiency, either being too simple to promote the student’s learning or too complex to be effective [4, 17, 30, 66]. These

Exercise								
Knowledge Concept 1								
Knowledge Concept 2								
Knowledge Concept 3								
Timestamp	1	2	4	8	11	12	17	20
Response								

Fig. 1. An example illustrating the KT task in intelligent education. Each column denotes an exercise at a timestamp, the colored circles in the three concept rows indicate the concepts associated with that exercise, and the bottom row records the observed response correctness. The goal is to predict whether the student will answer the final exercise correctly.

irregularities introduce spurious correlations that compromise model robustness, especially in large-scale deployments. While recent studies recognize data quality as critical for reliable KT [4, 30], they primarily focus on architectural refinements rather than mitigating the impact of noise on learning trajectories. A robust KT model should not only predict future performance but also distinguish genuine learning patterns from misleading fluctuations in interaction data [2, 19]. Effectively addressing this challenge requires a data-driven approach that accounts for the iterative nature of model training, enabling the model to adapt to both reliable and noisy learning signals.

- *Suboptimal data utilization efficiency.* From a data perspective, training efficiency is strongly influenced by how data is selected and used at each batch during learning, especially in diverse and noisy datasets. Existing KT models often rely on fixed sampling rules that ignore data distribution, sample quality, and the model’s current state [16, 21, 49, 54]. This causes inefficient data utilization, with valuable samples underused and noisy ones overused, hindering convergence and generalization. How to schedule the training data while considering the above factors still remains a fundamental challenge. An ideal solution should balance exploration, which samples diverse or uncertain data, and exploitation, which focuses on high-value samples, allowing the training to adapt to the evolving data-driven learning state of the model.

To address the above challenges, we propose a novel approach, termed **Reinforced Robust KT via Dynamic Graph Curriculum Learning (R²GCurL)**, which tackles the challenges from the perspectives of data structure design, noise robustness, and data efficiency. In particular, *to address the data structure design challenge*, we reformulate the KT task as a graph-level classification problem within a GNN framework. Instead of treating student response sequences as flat time series, we construct a graph at each timestep, where nodes represent previously attempted exercises or concepts, and edge attributes encode contextual information such as response timestamps or attempt frequencies. This transforms the raw sequential data into a dynamic graph, which is a temporally evolving structure composed of interrelated static graphs. Such data structure captures both the content and temporal dependencies within a student’s learning trajectory. By structuring the data in this way, we enable the GNN to exploit the latent relational patterns inherent in the sequence, enhancing its ability to model complex learning behaviors. The model then predicts the probability that a student will correctly answer the next exercise based on the graph representation of their prior interactions. *To address the challenge of robustness to noisy data*, we introduce a novel

Curriculum Learning (CL) framework into the KT field, which has been widely proven to be effective in noisy environments in extensive fields like computer vision and natural language processing [16, 49, 51]. At the core of our approach is a difficulty measurer specifically designed for dynamic graph-structured data. Under the current mathematical definition, the pairwise quantity is denoted as dynamic graph entropy. This dynamic graph entropy measures graph-transition continuity and structural consistency between graph states. Larger values indicate higher graph similarity and stronger structural consistency rather than higher uncertainty or larger structural variation. At the sequence level, these entropy quantities are aggregated into sequence-level dynamic graph entropy, and the curriculum module derives a structure-aware difficulty signal from them. Lower entropy means that the current exercise context is less aligned with nearby interactions or with the accumulated learning history, which tends to make the sequence harder for the model to fit reliably during training. This entropy-based signal is then used to modulate data selection under noisy and heterogeneous response patterns. *For the data efficiency challenge*, we employ **Reinforcement Learning (RL)** as a training scheduler to dynamically select the most suitable data samples for model training. Unlike conventional approaches that iteratively sample data without considering global correlations, our RL-based framework ensures that the training process adapts continuously, selecting samples that contribute to globally optimal learning efficiency. By balancing exploration and exploitation, our method improves convergence speed while maintaining high accuracy. Moreover, our approach is flexible enough to be easily adapted to various sequence-based KT models and datasets. Compared to other graph-based methods that build a large graph containing all exercises and concepts, our R^2GCuRL is also scalable, allowing it to handle large datasets effectively without significantly increasing complexity.

The main technical contributions of this study can be summarized as follows:

- (1) We formulate KT as dynamic graph modeling over student-specific exercise-concept evolution, enabling the model to capture both local interaction structure and cumulative historical structure within personalized learning trajectories.
- (2) We introduce a CL mechanism that combines a continuity-based structural signal derived from dynamic graph entropy with an RL-based scheduler, enabling the training process to distinguish more reliable from more disruptive trajectory patterns and to adapt curriculum emphasis through long-horizon sample scheduling under noisy and heterogeneous interactions.
- (3) We provide theoretical justification for the curriculum module from two perspectives: an information-theoretic analysis showing the utility advantage of structural representations for difficulty estimation, and a sequential decision-making analysis showing why the RL-based scheduler is better suited than myopic heuristic rules for adaptive curriculum optimization.
- (4) We conduct comprehensive empirical validation across predictive performance, scheduler necessity, computational efficiency, scalability, convergence behavior, entropy-pattern analysis, and trajectory-level knowledge-state behavior, showing that R^2GCuRL consistently improves KT robustness under noisy settings.

2 Related Work

2.1 KT Methods

Current deep KT research mainly improves predictive performance through sequence-based and graph-based architectures. Sequence-based KT models infer evolving learner states directly from the interaction sequence. DKT [41] first applies an RNN to hidden knowledge-state modeling, DKVMN [63] introduces an external key-value memory to maintain concept-related states, and attention-based models such as SAKT [34], AKT [15], and ELAKT [42] selectively emphasize

informative historical interactions when making predictions. These methods are strong at temporal dependency modeling, but the exercise–concept structure is usually encoded only implicitly in latent representations. Graph-based KT methods strengthen structural modeling by representing exercises, concepts, or their relations more explicitly. GKT [33] performs node-level reasoning on a predefined concept graph, GIKT [60] and PEBG [28] inject graph embeddings from static exercise–concept graphs into sequence models, DGEKT [10] adopts a hypergraph over exercises and concepts, and DyGKT [8] further introduces temporal graph dynamics. Despite these advances, much of the graph-based literature [22, 38, 39] still relies on global or relatively static structures shared across students, which limits the modeling of student-specific structural evolution over time. This is the gap targeted by the dynamic graph component of R²GCuRL, whose goal is to preserve both interaction order and personalized exercise–concept structure. In parallel, robust KT methods such as HD-KT [30] improve resilience to irregular historical responses caused by behaviors such as guessing or forgetting, but they do not explicitly address sequence-level heterogeneity or how training data should be adaptively scheduled under heterogeneous noise conditions. We therefore introduce CL [5] as a complementary data-centric mechanism for KT under noisy and heterogeneous training conditions.

2.2 CL Methods

Based on the principle of progressing from simple to complex training, CL was introduced to strategically organize model training [5, 13, 59, 64]. CL methods are generally categorized into predefined and automatic approaches. Both types initially assess the difficulty of training samples, and then schedule the training process accordingly. Predefined CL methods like Babystep [50] and OnePass [53] utilize discrete schedulers that categorize data into buckets sorted by increasing difficulty and start training with the simplest bucket. Babystep incrementally incorporates more difficult data, whereas OnePass discards its current bucket for a more difficult one at each transition. However, such abrupt changes between difficulty levels may introduce sharp learning gradients, potentially disrupting learning continuity and hindering the model’s generalization capabilities. In contrast, continuous schedulers like CurGraph [55] and MTDf [51] implement a smoother transition method, providing a gentle shift from easier to harder data samples, which can enhance model robustness and smooth the learning curve. Despite their benefits, these predefined methods often overlook model feedback, which is critical for optimizing the training process. Automatic CL methods like RLCuRL [20] and RCL [65] tackle this by splitting data samples into buckets and using a RL scheduler that dynamically selects training buckets based on ongoing model performance. These automatic methods, while effective at selecting one or multiple buckets based on model performance, may include samples in a bucket that are not suitable for the current training stage, potentially leading to reduced training efficiency. Our method mitigates this by introducing a RL algorithm that assigns a selection probability to each sample based on model performance, effectively balancing efficiency and performance throughout the training process.

3 Preliminary

In this section, we introduce some basic concepts about KT and then formulate the problem of KT. In addition, frequently used symbols are summarized in Table 1.

3.1 Basic Concepts

Exercises and Knowledge Concepts. In the domain of KT, educators commonly use exercises to assess students’ understanding of specific knowledge concepts. In this article, we denote the set of exercises as $Q = \{q_1, q_2, \dots, q_M\}$ and the set of all knowledge concepts as $C = \{c_1, c_2, \dots, c_K\}$. We

Table 1. Symbols and Definitions

Symbol	Definition
$G = (\mathcal{V}, \mathcal{E}, \mathbf{X})$	Exercise-concept bipartite graph
$\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$	Set of M exercises
$\mathcal{C} = \{c_1, c_2, \dots, c_K\}$	Set of K concepts
$\mathcal{V} = \mathcal{Q} \cup \mathcal{C}$	Set of nodes
$N = M + K$	Number of nodes in the graph
$\mathcal{E} \subseteq \mathcal{Q} \times \mathcal{C}$	Set of edges
$\mathbf{X} \in \mathbb{R}^{N \times d}$	Node feature matrix
$\mathcal{D} : \mathcal{Q} \rightarrow 2^{\mathcal{C}}$	The dictionary that each exercise $q_i \in \mathcal{Q}$ is mapped to a subset of related concepts $C_{q_i} \subseteq \mathcal{C}$
$R = [(q_1, r_1), \dots, (q_T, r_T)]$	A response sequence of length T
$R_i \in \mathcal{R}$	The i th response sequence in the dataset
$\mathcal{R} = \{R_1, R_2, \dots\}$	Set of response sequences
G_{Q_t}	Q -graph at timestep t
G_{S_t}	S -graph accumulated up to timestep t
$\mathcal{G}_{DQ} = [G_{Q_t}]_{t=1}^T$	Dynamic Q -graph
$\mathcal{G}_{DS} = [G_{S_t}]_{t=1}^T$	Dynamic S -graph
$N_1(v)$	The first-order neighbors of node v
$H(G_t, G_{t+1})$	Pairwise dynamic graph entropy between G_t and G_{t+1} ; larger values imply stronger graph-transition continuity
$H(\mathcal{G})$	Sequence-level dynamic graph entropy of graph sequence $\mathcal{G}$
$H(R_i)$	Structure-aware difficulty signal of the response sequence R_i derived from dynamic graph entropy
$H'(\mathcal{G}_{DQ}^i)$	Standardized dynamic Q -graph entropy of $\mathcal{G}_{DQ}^i$
$H'(\mathcal{G}_{DS}^i)$	Standardized dynamic S -graph entropy of $\mathcal{G}_{DS}^i$
λ	The hyper-parameter that weights the entropy-based difficulty signal
S_t	Multi-discrete state at step t
A_t	Multi-discrete action at step t
$Reward(S_t)$	Reward of state S_t
$p(x_i)$	The probability of data sample x_i being selected for the next training iteration

define a mapping function $\mathcal{D} : \mathcal{Q} \rightarrow 2^{\mathcal{C}}$, where each exercise $q_i \in \mathcal{Q}$ is associated with a subset of knowledge concepts $C_{q_i} \subseteq \mathcal{C}$, indicating the relevant knowledge concepts for that exercise.

Response Interactions. Students engage with exercises and receive feedback on their performance. We represent this sequence of exercises and results as the response interactions $R = [(q_1, r_1), (q_2, r_2), \dots, (q_T, r_T)]$, where $q_i \in \mathcal{Q}$ is an exercise, and $r_i \in \{0, 1\}$ is a binary indicator of correctness, with 1 indicating a correct response and 0 indicates the opposite.

Exercise-Concept Bipartite Graph. Given the set of exercises $\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$ and the set of concepts $\mathcal{C} = \{c_1, c_2, \dots, c_K\}$, where each exercise is associated with one or more knowledge concepts, we construct an exercise-concept bipartite graph denoted as $G = (\mathcal{V}, \mathcal{E}, \mathbf{X})$. Here, $\mathcal{V} = \mathcal{Q} \cup \mathcal{C}$ represents the set of nodes, $\mathcal{E} \subseteq \mathcal{Q} \times \mathcal{C}$ denotes the set of edges constructed based on the mapping function $\mathcal{D} : \mathcal{Q} \rightarrow 2^{\mathcal{C}}$, and $\mathbf{X} \in \mathbb{R}^{N \times d}$ represents the corresponding embeddings of all nodes, where each node's embedding is derived from the exercise and concept IDs [33, 60], $N = M + K$ denotes the number of nodes, and d is the dimension of node embeddings.

3.2 Problem Formulation

The goal of KT is to model and predict a student's knowledge state over time based on the historical response sequences. Therefore, we can formulate this problem as follows.

Definition 1 (KT). Given a set of exercises $\mathcal{Q}$, a set of related knowledge concepts $\mathcal{C}$, and a mapping function $\mathcal{D}$ that associates each exercise $q_i \in \mathcal{Q}$ with a subset of concepts $C_{q_i} \subseteq \mathcal{C}$, the goal is to

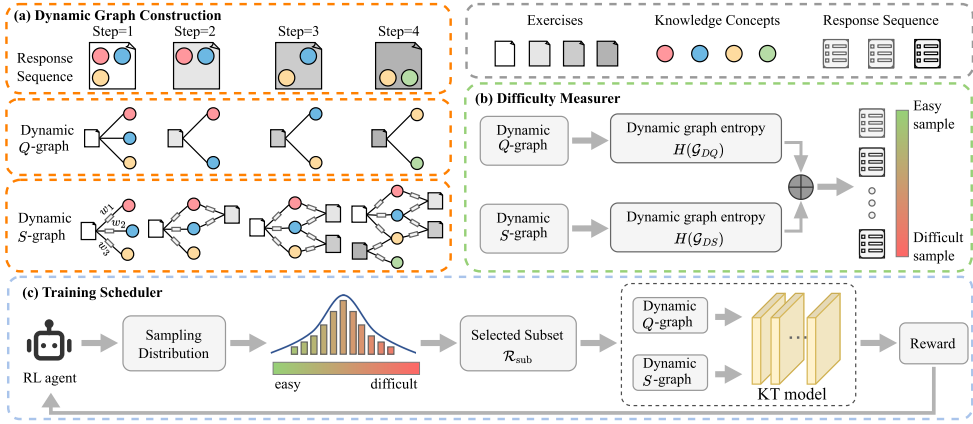


Fig. 2. An illustration of the proposed R²GCuRL framework. (a) Each response sequence is transformed into a dynamic *Q*-graph and a dynamic *S*-graph. (b) A structure-aware difficulty signal is induced from the dynamic graph entropy computed on these dynamic graphs. (c) An RL-based training scheduler and the KT model are jointly trained to select an optimal subset for the KT task in each iteration.

predict the probability $P(r_{T+1} = 1)$ that the student will correctly answer a future exercise q_{T+1} . This prediction is based on the student's sequence of response interactions $R = [(q_1, r_1), \dots, (q_T, r_T)]$.

4 Framework Description

As illustrated in Figure 2, R²GCuRL has two tightly connected components: dynamic graph modeling for personalized exercise–concept representation, and CL for adaptive data scheduling. The first addresses the representation problem in KT, while the second addresses data quality and training efficiency.

Dynamic Graph Modeling. Each response sequence is converted into two graph sequences. The dynamic *Q*-graph captures the local structure of the current exercise and its associated concepts at each step, whereas the dynamic *S*-graph accumulates the exercise–concept structure observed so far in the student's trajectory. Together, these two views combine local step-level structure with longer-range historical structure.

CL. We further introduce a curriculum mechanism to control how response sequences are presented during training. Instead of assuming that all sequences should be sampled uniformly throughout training, the framework derives a sequence-level signal and adapts the sampling distribution accordingly. (1) *Difficulty Measurer.* The difficulty measurer computes pairwise dynamic graph entropy from the dynamic *Q*-graph and dynamic *S*-graph and aggregates these values into sequence-level graph entropy scores. The pairwise dynamic graph entropy measures graph-transition continuity between graph states. Larger H indicates stronger structural continuity rather than larger structural variation or higher model-side uncertainty. The curriculum module therefore derives a structure-aware difficulty signal from these entropy values, rather than equating entropy itself with difficulty. Lower entropy implies that the current exercise context is less aligned with nearby interactions or with the accumulated learning history, which tends to make the sequence harder for the model to fit reliably. (2) *RL-Based Training Scheduler.* Based on this entropy-based signal, the RL-based scheduler adaptively adjusts the sampling distribution during training. Its role is to balance exploitation of informative regions of the curriculum with exploration of neighboring difficulty levels, so that the KT model can respond to heterogeneous and evolving training needs. Its empirical benefit relative to simpler schedulers is evaluated in the experiments section. The

scheduler serves as an adaptive data selection mechanism that controls curriculum emphasis during training, while the KT model remains the predictive component.

Overall, R²GCuRL unifies personalized structural modeling and adaptive data scheduling within a single KT framework.

5 Methodology

In this section, we introduce our R²GCuRL framework for KT, which first constructs dynamic graphs based on students' response sequences and then trains the KT model and CL modules together in an end-to-end fashion to complete the KT task. In particular, we first explain the construction of dynamic graphs and the graph modeling module in Section 5.1. Next, we explain the CL pipeline in Section 5.2, including the difficulty measurer based on dynamic graph entropy and the RL-based training scheduler.

5.1 Dynamic Graph Modeling

Motivation. In the field of KT, students' learning behaviors and knowledge states are dynamically evolving. Sequence-based KT models fail to capture the complex relationships between exercises and concepts. In contrast, current graph-based KT models usually learn from static graphs, which fail to capture this dynamic nature. Therefore, we introduce a dynamic graph modeling approach to bridge this gap. By constructing and utilizing dynamic exercise–concept bipartite graphs, we reformulate the KT task as a graph classification task. This approach can be extended to current sequence-based KT models and enhance their performance by adapting the module to model the complex relationships between exercises and concepts.

5.1.1 Exercise–Concept Bipartite Graph Construction. Given the exercise set $Q = \{q_1, \dots, q_M\}$, the concept set $C = \{c_1, \dots, c_K\}$, the response sequence $R = [(q_1, r_1), (q_2, r_2), \dots, (q_T, r_T)]$, and the exercise–concept mapping $\mathcal{D} : Q \rightarrow 2^C$, we build a graph at every step $1 \leq t \leq T$. At step t , the Q -graph contains only the current exercise q_t and its associated concepts, so it provides a local structural view of the current interaction. The S -graph, by contrast, aggregates all exercises observed from step 1 to step t together with their associated concepts, giving a cumulative structural view of the student's learning history up to that point. The dynamic Q -graph therefore captures step-wise structural changes, whereas the dynamic S -graph describes how the exercise–concept structure evolves over time for an individual student.

In the Q -graph, we model the connections between exercises and knowledge concepts using an undirected bipartite graph. Specifically, at each step t , a Q -graph G_{Q_t} is constructed to link the current exercise node to its associated knowledge concept nodes. Formally, a Q -graph can be defined as:

Definition 2 (Q-Graph). Given the exercise q_t at step t and its concept set $C_{q_t} = \mathcal{D}(q_t)$, the Q -graph at step t can be formulated as $G_{Q_t} = (\mathcal{V}_{Q_t}, \mathcal{E}_{Q_t}, \mathbf{X}_{Q_t})$, where $\mathcal{V}_{Q_t} = \{q_t\} \cup C_{q_t}$, $\mathcal{E}_{Q_t} = \{(q_t, c) : c \in C_{q_t}\}$, and $\mathbf{X}_{Q_t} \in \mathbb{R}^{|\mathcal{V}_{Q_t}| \times d}$ represents the node feature matrix.

In the S -graph, we construct a cumulative bipartite graph from the prefix of a response sequence $R = [(q_1, r_1), (q_2, r_2), \dots, (q_T, r_T)]$ up to step t . Each exercise node is linked to its relevant concepts, and edges are weighted to reflect the count of correct and incorrect responses, accommodating repeated exercises within the sequence. Formally, the S -graph can be defined as follows:

Definition 3 (S-Graph). Given a response sequence $R = [(q_1, r_1), (q_2, r_2), \dots, (q_T, r_T)]$, the S -graph accumulated up to step t can be formulated as $G_{S_t} = (\mathcal{V}_{S_t}, \mathcal{E}_{S_t}, \mathbf{X}_{S_t})$, where $\mathcal{V}_{S_t} = \{q_1, q_2, \dots, q_t\} \cup \bigcup_{\tau=1}^t C_{q_\tau}$, $\mathcal{E}_{S_t} = \{(q_\tau, c, w_{q_\tau, c}) : 1 \leq \tau \leq t, c \in C_{q_\tau}\}$, and $\mathbf{X}_{S_t} \in \mathbb{R}^{|\mathcal{V}_{S_t}| \times d}$ represents the node feature

Algorithm 1: Dynamic Graph Construction

Require: Response sequence $R = [(q_1, r_1), (q_2, r_2), \dots, (q_T, r_T)]$, exercise–concept mapping $\mathcal{D} : Q \rightarrow 2^C$.

Ensure: Dynamic Q -graph sequence $\mathcal{G}_{DQ}$, dynamic S -graph sequence $\mathcal{G}_{DS}$

```

1: Initialize  $\mathcal{G}_{DQ} \leftarrow []$ ,  $\mathcal{G}_{DS} \leftarrow []$ 
2: Initialize cumulative node set  $\mathcal{V}_S \leftarrow \emptyset$ , cumulative edge set  $\mathcal{E}_S \leftarrow \emptyset$ 
3: for  $t = 1$  to  $T$  do
4:   // Construct  $Q$ -graph  $G_{Q_t}$  for step  $t$ 
5:   Retrieve current exercise  $q_t$  and its concepts  $C_{q_t} = \mathcal{D}(q_t)$ 
6:    $\mathcal{V}_{Q_t} \leftarrow \{q_t\} \cup C_{q_t}$ 
7:    $\mathcal{E}_{Q_t} \leftarrow \{(q_t, c) : c \in C_{q_t}\}$ 
8:   Initialize node feature matrix  $\mathbf{X}_{Q_t}$  for  $\mathcal{V}_{Q_t}$ 
9:    $G_{Q_t} \leftarrow (\mathcal{V}_{Q_t}, \mathcal{E}_{Q_t}, \mathbf{X}_{Q_t})$ 
10:  Append  $G_{Q_t}$  to  $\mathcal{G}_{DQ}$ 
11:  // Construct  $S$ -graph  $G_{S_t}$  up to step  $t$ 
12:   $\mathcal{V}_{S_t} \leftarrow \mathcal{V}_S \cup \{q_t\} \cup C_{q_t}$ 
13:   $\mathcal{E}_{S_t} \leftarrow \mathcal{E}_S$ 
14:  for each concept  $c \in C_{q_t}$  do
15:    Update edge weight  $w_{q_t, c}$  in  $\mathcal{E}_{S_t}$  based on response  $r_t$ 
16:  end for
17:  Initialize node feature matrix  $\mathbf{X}_{S_t}$  for  $\mathcal{V}_{S_t}$ 
18:   $G_{S_t} \leftarrow (\mathcal{V}_{S_t}, \mathcal{E}_{S_t}, \mathbf{X}_{S_t})$ 
19:   $\mathcal{V}_S \leftarrow \mathcal{V}_{S_t}$ ,  $\mathcal{E}_S \leftarrow \mathcal{E}_{S_t}$ 
20:  Append  $G_{S_t}$  to  $\mathcal{G}_{DS}$ 
21: end for
22: return  $\mathcal{G}_{DQ}, \mathcal{G}_{DS}$ 

```

matrix. The edges in $\mathcal{E}_{S_t}$ are weighted, with $w_{q_t, c}$ recording the counts of correct and incorrect responses associated with exercise–concept pair (q_t, c) .

This approach captures the structural information within exercises and concepts. The Q -graph reflects the structural information for a single exercise and its corresponding concepts, while the S -graph reflects the structural information for all exercises and their corresponding concepts within a given response sequence R .

5.1.2 Running Example of the Graph Construction Process. To provide a more intuitive understanding of the construction process (Algorithm 1), consider a concrete running example. Suppose a student interacts with two exercises: at $t = 1$, they answer exercise q_1 associated with concepts c_1 and c_2 ; at $t = 2$, they answer q_2 associated with c_2 and c_3 . At $t = 1$, the Q -graph G_{Q_1} connects q_1 to c_1 and c_2 , and the S -graph G_{S_1} is identical. At $t = 2$, the Q -graph G_{Q_2} connects only q_2 to c_2 and c_3 , while the S -graph G_{S_2} aggregates the history and includes all nodes $\{q_1, q_2, c_1, c_2, c_3\}$. This cumulative structure explicitly captures that concept c_2 has been practiced across multiple exercises, while edge weights record cumulative correctness. Table 2 extends this toy example to three interactions and lists the resulting Q -graph and S -graph edges.

5.1.3 Graph Modeling Module. To retain the GNN advantages, we adopt the same mechanism of node feature learning in the existing GNNs with multi-layer local aggregation. We apply a standard GNN for the Q -graph and extend it to WeightGNN for encoding the S -graph with edge

Table 2. Running Example of Dynamic Q-Graph and S-Graph Construction on a Toy Sequence

t	Interaction (q_t, r_t)	C_{q_t}	Q-graph edges	S-graph weighted edges
1	$(q_1, 1)$	$\{c_1\}$	$\{(q_1, c_1)\}$	$\{(q_1, c_1) : (1, 0)\}$
2	$(q_2, 0)$	$\{c_1, c_2\}$	$\{(q_2, c_1), (q_2, c_2)\}$	$\{(q_1, c_1) : (1, 0); (q_2, c_1) : (0, 1); (q_2, c_2) : (0, 1)\}$
3	$(q_3, 1)$	$\{c_3\}$	$\{(q_3, c_3)\}$	$\{(q_1, c_1) : (1, 0); (q_2, c_1) : (0, 1); (q_2, c_2) : (0, 1); (q_3, c_3) : (1, 0)\}$

Each weighted edge in the S-graph is written as (n^+, n^-) , denoting the cumulative numbers of correct and incorrect responses, respectively.

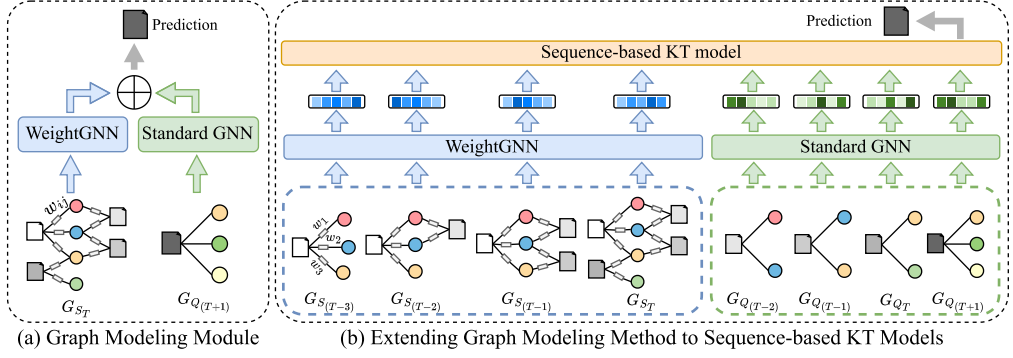


Fig. 3. An illustration of (a) the graph modeling module, and (b) extending the graph modeling method for sequence-based KT models. Given the constructed graphs, a standard GNN and the weightGNN are employed to model the structural information.

weights (see Figure 3(a)). WeightGNN has L local aggregation layers, allowing each node to access information from L -hop neighbors. Given the S-graph $G_S = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, the calculation for the l th layer of WeightGNN is:

$$\mathbf{m}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{h}_u^{(l-1)} \parallel \phi^{(l-1)}(w_{vu}) : u \in \mathcal{N}_1(v) \right\} \right), \quad (1)$$

$$\mathbf{h}_v^{(l)} = \text{UPDATE}^{(l)} \left(\mathbf{h}_v^{(l-1)}, \mathbf{m}_v^{(l)} \right), \quad (2)$$

where $\mathbf{h}_v^{(l)}$ is the feature of node v at layer l , $\text{AGG}^{(l)}$ and $\text{UPDATE}^{(l)}$ represent the aggregation and update functions, respectively, and $\phi^{(l-1)}(w_{vu})$ embeds edge weights. For the unweighted Q-graph, a standard GNN (e.g., GCN, GAT) is used for structural learning.

Given S-graph $G_S = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, we compute node embeddings with WeightGNN and obtain the graph embedding with READOUT. For simplicity, we provide the calculation for the S-graph below, and the calculation for the Q-graph is analogous:

$$\mathbf{H} = \text{WeightGNN}(\mathcal{V}, \mathcal{E}, \mathbf{X}), \quad (3)$$

$$\mathbf{h}_G = \text{READOUT}(\mathbf{H}), \quad (4)$$

where $\mathbf{H}$ is the node embeddings, $\text{READOUT}(\cdot)$ represents a pooling function (e.g., mean and max pooling), and $\mathbf{h}_G$ is the calculated graph embedding. Next, the graph embeddings of S-graph and Q-graph are input into a classifier to obtain the prediction results:

$$P_{T+1} = \text{Classifier}(\mathbf{h}_{G_{S_T}} \parallel \mathbf{h}_{G_{Q_{T+1}}}), \quad (5)$$

where P_{T+1} is the prediction for the $(T + 1)$ th exercise, where the classifier can be an MLP, and $\mathbf{h}_{G_{S_T}}$ and $\mathbf{h}_{G_{Q_{T+1}}}$ are the graph embeddings for the S -graph and Q -graph, respectively. Specifically, the $(T + 1)$ th Q -graph is used as the query for prediction because the T th S -graph lacks the information of the target exercise.

5.1.4 Extending Graph Modeling Module to Sequence-Based KT Models. Since the backbone of sequence-based KT models (e.g., RNN [56] and Transformer [52]) takes the response sequence R as input, we extend them with the graph modeling module by constructing the dynamic Q -graph and the dynamic S -graph, as illustrated in Figure 3(b).

Definition 4 (Dynamic Q -graph). Given a response sequence $R = [(q_1, r_1), (q_2, r_2), \dots, (q_T, r_T)]$ and the Q -graph definition in Definition 2, the dynamic Q -graph is defined as $\mathcal{G}_{DQ} = [G_{Q_t}]_{t=1}^T$, where each graph $G_{Q_t} = (\mathcal{V}_{Q_t}, \mathcal{E}_{Q_t}, \mathbf{X}_{Q_t})$ is the Q -graph at timestep t .

Definition 5 (Dynamic S -graph). Given a response sequence $R = [(q_1, r_1), (q_2, r_2), \dots, (q_T, r_T)]$ and the S -graph definition in Definition 3, the dynamic S -graph is defined as $\mathcal{G}_{DS} = [G_{S_t}]_{t=1}^T$, where each graph $G_{S_t} = (\mathcal{V}_{S_t}, \mathcal{E}_{S_t}, \mathbf{X}_{S_t})$ is the cumulative S -graph up to timestep t .

Constructing the dynamic G -graph and dynamic S -graph makes it flexible to introduce the structural information to the sequence-based KT models. For $\mathcal{G}_{DQ}$ and $\mathcal{G}_{DS}$, we obtain the corresponding graph-embedding sequences via Equations (3) and (4), denoted as $\mathbf{h}_{\mathcal{G}_{DQ}} = [\mathbf{h}_{G_{Q_t}}]_{t=1}^T$ and $\mathbf{h}_{\mathcal{G}_{DS}} = [\mathbf{h}_{G_{S_t}}]_{t=1}^T$, respectively. Let SeqKT represent the sequence-based KT model, the final prediction is then given by $P_{T+1} = \text{Classifier}(\text{SeqKT}(\mathbf{h}_{\mathcal{G}_{DQ}}, \mathbf{h}_{\mathcal{G}_{DS}}))$. The graph modeling module and the sequence-based KT model can be trained end-to-end using backpropagation.

5.2 CL

Motivation. To improve robustness under heterogeneous KT data, we adopt a CL pipeline with two components: a difficulty measurer and a training scheduler. The difficulty measurer provides sequence-level dynamic graph entropy derived from pairwise graph entropy on the dynamic graphs. This dynamic graph entropy measures continuity-oriented structural consistency between graph states rather than serving as a perfect or exhaustive definition of sample noise. The curriculum module converts these entropy values into a structure-aware difficulty signal. The training scheduler then adapts the sampling distribution according to the current training state. Its role is to balance exploration and exploitation while revisiting different regions of the curriculum signal as model needs evolve. Together, these components provide a data-centric training mechanism for KT under noisy and diverse response patterns.

5.2.1 Difficulty Measurer. In KT, the training difficulty of a response sequence is hard to define from raw response tokens alone, because they do not explicitly encode the underlying exercise–concept structure. We therefore compute dynamic graph entropy from the dynamic Q -graph and dynamic S -graph, which makes such structure explicit. $H(G_t, G_{t+1}) = 1 - \text{JS}(\cdot)$ measures graph-transition continuity between adjacent graph states, and larger values indicate stronger structural continuity and higher similarity. The dynamic Q -graph entropy emphasizes local continuity between consecutive interactions, whereas the dynamic S -graph entropy emphasizes how well each step remains aligned with the accumulated learning history. We standardize these sequence-level entropy values and use them to construct a structure-aware difficulty signal for CL. Difficulty is therefore not the entropy itself. It is derived from the entropy pattern across the response sequence. Lower entropy suggests weaker continuity between the current exercise context and nearby and accumulated structure, and this tends to make the sequence harder for the model to fit reliably during training.

Given a dynamic Q -graph $\mathcal{G}_{DQ}$ and a dynamic S -graph $\mathcal{G}_{DS}$ of the response sequence R , we first compute continuity-oriented dynamic graph entropy by leveraging multisets, i.e., sets with possibly repeated elements, together with the **Jensen-Shannon (JS)** divergence [26]. To clearly illustrate this computation, we employ the exercise-concept bipartite graph defined in Section 3.1, where the S -graph and Q -graph are treated as specific instances of this graph, thereby maintaining generality without redundant notation. The calculation process for a sequence of exercise-concept bipartite graphs, denoted as $\mathcal{G} = [G_t : t = 1, 2, \dots, T]$, is given below.

Step 1: Extracting Multisets. For each graph $G_t = (\mathcal{V}_t, \mathcal{E}_t, \mathbf{X}_t)$ in $\mathcal{G}$, we first derive its multiset: $\mathcal{S}_{G_t} = \{ \{(q, c) : (q, c) \in \mathcal{E}_t\}, \{q : q \in \mathcal{Q}_t\}, \{c : c \in \mathcal{C}_t\} \}$, where $\mathcal{S}_{G_t}$ is the multiset of G_t , (q, c) is the Cartesian product of the exercise and the concept, $q \in \mathcal{Q}_t$ is the exercise, and $c \in \mathcal{C}_t$ is the concept. Based on the definition of multiset, $\mathcal{S}_{G_t}$ can be rewritten as an ordered pair $\mathcal{M}_{G_t} = \{(\mathcal{A}_{G_t}, f_{G_t})\}$, where $\mathcal{A}_{G_t}$ is the set derived from distinct elements of $\mathcal{S}_{G_t}$, and $f_{G_t} : \mathcal{A}_{G_t} \rightarrow \mathbb{Z}^+$ is a function mapping each element in $\mathcal{A}_{G_t}$ to a positive integer representing its multiplicity, i.e., the number of times the element appears in the multiset.

Step 2: Computing Pairwise Graph Entropy. Given the ordered pair multiset $\mathcal{M}_{G_t}$ and $\mathcal{M}_{G_{t+1}}$, we first obtain the unique element set $\mathcal{A}_{G_{t+1}} = \mathcal{A}_{G_t} \cup \mathcal{A}_{G_{t+1}}$. By the definition of JS divergence, we first calculate the KL divergence $D_{KL}(G_t \parallel \frac{G_t + G_{t+1}}{2})$ from the graph G_{t+1} to the graph G_t , which is calculated as follows:

$$N(G_t) = [f_{G_t}(a_1), \dots, f_{G_t}(a_{N_a})] \in \mathbb{Z}^{N_a}, a_i \in \mathcal{A}_{G_{t+1}}$$

$$p(G_t) = [p_{G_t}^{a_1}, \dots, p_{G_t}^{a_{N_a}}] = \left[\frac{f_{G_t}(a_1)}{\sum_{i=1}^{N_a} f_{G_t}(a_i)}, \dots, \frac{f_{G_t}(a_{N_a})}{\sum_{i=1}^{N_a} f_{G_t}(a_i)} \right] \quad (6)$$

$$D_{KL} \left(G_t \parallel \frac{G_t + G_{t+1}}{2} \right) = \sum_{i=1}^{N_a} p_{G_t}^{a_i} \log \left(\frac{p_{G_t}^{a_i}}{\frac{p_{G_t}^{a_i} + p_{G_{t+1}}^{a_i}}{2}} \right), \quad (7)$$

where $N(G_t)$ is a list that each element is the number of a_i in $\mathcal{M}_{G_t}$, obtained by the function f_{G_t} . $N_a = |\mathcal{A}_{G_{t+1}}|$ denotes the size of unique element set; $\sum_{i=1}^{N_a} f_{G_t}(a_i)$ is the sum of the sequences $N(G_t)$; $p(G_t)$ is the normalized result of $N(G_t)$.

Then the pairwise graph entropy between the graph G_t and G_{t+1} can be calculated based on $D_{KL}(G_t \parallel \frac{G_t + G_{t+1}}{2})$ and $D_{KL}(G_{t+1} \parallel \frac{G_t + G_{t+1}}{2})$ as follows:

$$H(G_t, G_{t+1}) = 1 - \frac{1}{2} \left(D_{KL} \left(G_t \parallel \frac{G_t + G_{t+1}}{2} \right) + D_{KL} \left(G_{t+1} \parallel \frac{G_t + G_{t+1}}{2} \right) \right), \quad (8)$$

where $D_{KL}(G_t \parallel \frac{G_t + G_{t+1}}{2})$ represents the relative entropy from the graph G_{t+1} to the graph G_t , and $D_{KL}(G_{t+1} \parallel \frac{G_t + G_{t+1}}{2})$ is the opposite; $H(G_t, G_{t+1})$ is symmetric and denotes the pairwise graph entropy between them. Under Equation (8), this graph entropy measures graph-transition continuity between the two graphs, and larger values suggest higher similarity and stronger continuity.

Step 3: Computing Sequence-Level Dynamic Graph Entropy. Based on Equation (8), the sequence-level dynamic graph entropy for a sequence of exercise-concept bipartite graphs $\mathcal{G} = [G_t : t = 1, 2, \dots, T]$ is defined as:

$$H(\mathcal{G}) = \frac{1}{T-1} \sum_{t=1}^{T-1} H(G_t, G_{t+1}). \quad (9)$$

Hence, given a dynamic Q -graph $\mathcal{G}_{DQ}$ and a dynamic S -graph $\mathcal{G}_{DS}$ for a response sequence R , we calculate the dynamic Q -graph entropy and dynamic S -graph entropy as $H(\mathcal{G}_{DQ})$ and $H(\mathcal{G}_{DS})$, respectively.

For a set of response sequences as $\mathcal{R} = \{R_1, R_2, \dots, R_{N_R}\}$ that contains N_R elements, the dynamic Q -graph entropy and dynamic S -graph entropy are calculated for each sequence. Since the values of the two results vary significantly, we employ standardization for each before computing the entropy-based curriculum signal. This process ensures that the measurements are on a comparable scale.

Step 4: Computing the Structure-Aware Difficulty Signal. Given the dynamic Q -graph entropy $H(\mathcal{G}_{DQ}^i)$ for the response sequence R_i , the standardization can be performed as follows:

$$\begin{aligned}\mu_{DQ} &= \frac{1}{N_R} \sum_{i=1}^{N_R} H(\mathcal{G}_{DQ}^i) \\ \sigma_{DQ} &= \sqrt{\frac{1}{N_R} \sum_{i=1}^{N_R} (H(\mathcal{G}_{DQ}^i) - \mu_{DQ})^2} \\ H'(\mathcal{G}_{DQ}^i) &= \frac{H(\mathcal{G}_{DQ}^i) - \mu_{DQ}}{\sigma_{DQ}},\end{aligned}\tag{10}$$

where $H'(\mathcal{G}_{DQ}^i)$ is the standardized dynamic Q -graph entropy of $\mathcal{G}_{DQ}^i$ for the response sequence R_i ; μ_{DQ} and σ_{DQ} are the mean value and the standard deviation of the dynamic Q -graph entropy of $\mathcal{G}_{DQ}$ for all sequences $\mathcal{R}$, respectively. The standardization process of the dynamic S -graph entropy is similar, and we can obtain the standardized value $H'(\mathcal{G}_{DS}^i)$.

We then combine the standardized dynamic Q -graph entropy and dynamic S -graph entropy into the structure-aware difficulty signal of the response sequence R_i . Formally,

$$H(R_i) = -[(1 - \lambda)H'(\mathcal{G}_{DQ}^i) + \lambda H'(\mathcal{G}_{DS}^i)],\tag{11}$$

where $H(R_i)$ represents the structure-aware difficulty signal of the response sequence R_i ; $H'(\mathcal{G}_{DQ}^i)$ and $H'(\mathcal{G}_{DS}^i)$ denote the standardized dynamic graph entropy values of $\mathcal{G}_{DQ}^i$ and $\mathcal{G}_{DS}^i$, respectively; and λ is a hyper-parameter that controls the ratio.

5.2.2 Training Scheduler. In this section, we introduce an innovative RL strategy to optimize the training sample selection process in CL. Each training sample is mapped onto a one-dimensional curriculum axis using the entropy-based structure-aware difficulty signal. We then use a normal distribution on this axis to assign selection probabilities to each sample, where samples closer to the mean of the distribution have a higher probability of being selected, and those further from the center have a lower probability. Compared to selecting from pre-divided batches or using a rule-based training scheduler, this soft selection method based on a normal distribution enhances sample diversity at each training step. It is intended to better match the model's learning needs while preserving smooth curriculum transitions. The training scheduling process is modeled as a finite horizon **Markov Decision Process (MDP)**, which is characterized by a tuple $(\mathcal{S}, \mathcal{A}, P, \text{Reward}, \gamma)$, where $S_t \in \mathcal{S}$, $A_t \in \mathcal{A}$ are state and action observed at step t , P is a state transition function, Reward is a reward function, and γ is a discount factor. The training scheduling process is defined as follows:

- *State:* The state S_t at step t is defined as $S_t = [\mu_t, \sigma_t^2]$, where $0 \leq \mu_t \leq 1$ and $0.01 \leq \sigma_t^2 \leq 0.1$ represent the mean and variance of the normal distribution that governs the selection probabilities of training samples. The samples are mapped onto a scale from 0 to 1 on the

horizontal axis according to the curriculum signal. Initially, when $t=0$, $S_0 = [0.0, 0.01]$, with the mean set at 0 to start from one boundary of the curriculum axis, and a variance of 0.01 to allow exploration across adjacent difficulty levels.

- *Action*: The action A_t taken at step t involves adjusting the parameters of the normal distribution, represented as $A_t = [\Delta\mu, \Delta\sigma^2]$, where $\Delta\mu \in \{-0.1, 0, 0.1\}$ and $\Delta\sigma^2 \in \{-0.01, 0, 0.01\}$ are the changes applied to the mean μ_t and the variance σ_t^2 , respectively.
- *State Transition*: The state transition function updates the distribution parameters based on the actions taken:

$$S_{t+1} = P(A_t|S_t) = [\mu_{t+1}, \sigma_{t+1}^2] = [\mu_t + \Delta\mu, \sigma_t^2 + \Delta\sigma^2]. \quad (12)$$

Based on the updated distribution represented by state S_{t+1} , the data selection module would sample a batch of data for the next iteration of model training.

- *Reward*: The goal of the RL agent is to find the suitable sample distribution of the data samples for the KT model. A discrete reward function is defined based on the KT model performance:

$$\text{Reward}(S_t) = \mathcal{L}_{t-1} - \mathcal{L}_t, \quad (13)$$

where $\mathcal{L}_t$ represents the loss on the validation set at step t , and the reward Reward_t is calculated as the reduction in loss from one training iteration to the next. A decrease in loss, indicative of improved model performance, results in a positive reward. This mechanism ensures that the reward increases as the model shows more significant improvements from one batch of samples to the next. It is important to note that this reward function can be replaced with alternative metrics as long as they guide the RL agent to select the appropriate distribution for data selection.

- *Data Selection Module*: An illustration of the module is shown in Figure 4. Given a total of N_R samples, each sample x_i is assigned a position z_i on a normalized scale from 0 to 1: $z_i = \frac{i}{N_R-1}$, where $i = 0, 1, 2, \dots, N_R - 1$. We then calculate initial scores for each sample using the normal distribution with μ_t and σ_t^2 :

$$\text{Score}(x_i) = \frac{1}{\sqrt{2\pi\sigma_t^2}} \exp\left(-\frac{(z_i - \mu_t)^2}{2\sigma_t^2}\right). \quad (14)$$

To ensure valid sampling, the selection values are normalized so that their sum across all training samples equals 1:

$$p(x_i) = \frac{\text{Score}(x_i)}{\sum_{j=0}^{N_R-1} \text{Score}(x_j)}. \quad (15)$$

Based on the probabilities $p(x_i)$, we utilize weighted random sampling to select a fixed number of data samples, denoted as $\mathcal{R}_{\text{sub}}$, from the entire set of response sequences $\mathcal{R}$ for the next iteration of model training. The KT model is then trained on these selected data samples, and the resulting loss is used to compute the reward according to Equation (13).

During training, the KT model and the RL-based scheduler are optimized jointly, but they play different roles. The KT model remains the predictive component, whereas the scheduler only controls which part of the curriculum is emphasized at each iteration. As shown in Figure 5, the KT model first evaluates the sampled batch and produces the validation feedback used to compute reward. The RL agent then updates its policy and adjusts the next sampling distribution through the state transition in Equation (12). This loop allows the scheduler to reweight the curriculum adaptively according to model feedback. In this sense, the RL module acts as an adaptive data-selection mechanism for optimization rather than as an additional knowledge-state predictor.

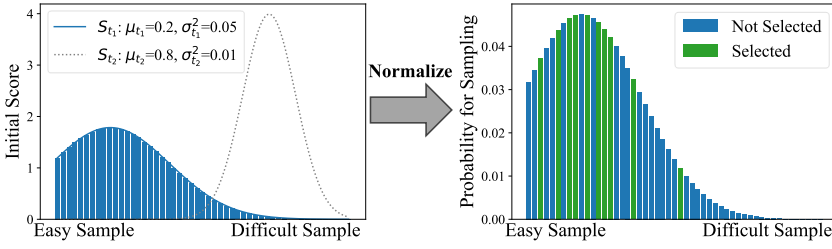


Fig. 4. An illustration of the data selection module. Different states correspond to different sampling distributions. For example, consider the state $S_{t_1} = (\mu_{t_1}, \sigma_{t_1}^2) = (0.2, 0.05)$, the module calculates initial scores for each sample, based on the specified distribution parameters. These scores are then converted into probabilities that determine each sample's likelihood of selection. This probability guides the weighted random sampling process used to select a fixed number of samples for the subsequent training iteration. In the histogram below, the green bars represent the selected data samples, while the blue bars denote those that are not selected.

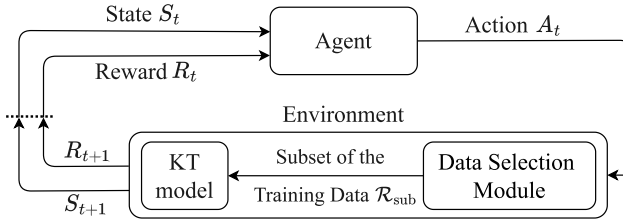


Fig. 5. The agent environment interaction in RL.

6 Model Training

In Algorithm 2, we present a detailed procedure of our training framework. The process begins by calculating the $H(\cdot)$ -based dynamic graph entropy for each response sequence based on the defined dynamic Q -graph and dynamic S -graph (lines 1–6). This dynamic graph entropy measures graph-transition continuity and structural consistency between graph states. Subsequently, all response sequences are arranged according to $H(R_i)$ (line 7). Lastly, the KT model and the RL-based training scheduler are trained together in an end-to-end manner (lines 8–15). At each step t , the RL agent selects a subset $\mathcal{R}_{\text{sub}}$ from the training dataset for training the KT model. The KT model is then evaluated on the validation set to compute the loss, which is utilized to calculate the reward. This feedback is used to update the RL policy network, thereby refining the training strategy iteratively.

7 Theoretical Analysis

In this section, we provide a theoretical analysis of the proposed R²GCurl, focusing on the difficulty signal and the RL-based scheduler in the CL module. We also theoretically compare the complexity of R²GCurl with that of state-of-the-art models. For clarity of presentation, we slightly abuse notation where appropriate.

7.1 Theoretical Justification of the CL Module

The theoretical justification of the curriculum module is organized around two propositions. The first concerns the representation underlying the difficulty measurer, and the second concerns the RL-based scheduler under a fixed difficulty measurer. Our goal is not to claim that a particular scalar metric is uniquely optimal, but to clarify why (i) structural representations can provide strictly

Algorithm 2: The Overall Process of Training R²GCuRL

Require: $\mathcal{R}=\{R_1, R_2, \dots, R_{N_R}\}$
 // Calculate dynamic graph entropy
 1: **for** $R_i \in \mathcal{R}$ **do**
 2: Construct dynamic Q-graph $\mathcal{G}_{DQ}^i$
 3: Construct dynamic S-graph $\mathcal{G}_{DS}^i$
 4: Calculate standardized dynamic graph entropy $H'(\mathcal{G}_{DQ}^i)$ and $H'(\mathcal{G}_{DS}^i)$
 5: Calculate the entropy-based curriculum signal $H(R_i)$
 6: **end for**
 7: Sort the data samples according to $H(R_i)$
 // Train the KT model and the RL-based training scheduler jointly in an end-to-end manner
 8: **for** step=1,2,... **do**
 9: The RL agent selects a subset of the dataset $\mathcal{R}_{\text{sub}}$ for KT model training via Eq. (15)
 10: Train the KT model with backpropagation on $\mathcal{R}_{\text{sub}}$
 // RL process
 11: Calculate the loss $\mathcal{L}_t$ on the validation set
 12: $\text{Reward}(S_t) \leftarrow \text{Eq. (13)}$
 13: $A_t \leftarrow \text{RL policy network}$
 14: $S_{t+1} \leftarrow P(A_t|S_t)$ via Eq. (12)
 15: **end for**

richer information for curriculum-relevant training decisions, and (ii) an RL-based scheduler is better suited than myopic heuristic rules when curriculum decisions have long-horizon effects.

7.1.1 Analysis on the Difficulty Signal. We analyze the difficulty signal from an information-theoretic perspective. The key issue is not whether a metric exactly matches an abstract notion of difficulty, but whether the representation from which the metric is constructed preserves information relevant to training decisions.

Let $\mathcal{V}(\theta)$ denote the validation loss, where smaller values indicate better generalization, and let $\mathcal{L}(R; \theta)$ denote the training loss induced by response sequence R under model parameters θ . Starting from model state θ_t , one gradient-descent update on R gives $\theta_{t+1}(R) = \theta_t - \eta \nabla_{\theta} \mathcal{L}(R; \theta_t)$. We define the one-step training utility of R at step t as $U_t(R) = \mathcal{V}(\theta_t) - \mathcal{V}(\theta_{t+1}(R))$. Thus, $U_t(R)$ measures how much a single update on R reduces validation loss. Assuming that $\mathcal{V}$ is continuously differentiable in a neighborhood of θ_t , a first-order Taylor expansion gives $U_t(R) = \eta \langle \nabla \mathcal{V}(\theta_t), \nabla_{\theta} \mathcal{L}(R; \theta_t) \rangle + \mathcal{O}(\eta^2)$. Therefore, $U_t(R)$ is the local quantity governing the immediate usefulness of a sample for optimization, and it is a natural target for utility-aware curriculum ranking.

To compare candidate difficulty signals, we compare how much information their underlying representations retain about $U_t(R)$. For a response sequence R , let $X_Q(R)$ denote a question-only representation and $X_C(R)$ a concept-only representation. Let $X_{\text{seq}}(R) = (X_Q(R), X_C(R))$ denote a direct sequence representation that preserves question and concept identities but not their structural relations. Let $X_{\text{str}}(R) = (X_{\text{seq}}(R), X_{\text{loc}}(R), X_{\text{cum}}(R))$ denote a structural representation, where $X_{\text{loc}}(R)$ and $X_{\text{cum}}(R)$ encode local and cumulative exercise-concept structure, respectively.

The following assumption formalizes that structural information contributes to one-step training utility beyond raw sequence identity.

ASSUMPTION 1. $I(U_t; X_{\text{loc}}, X_{\text{cum}} \mid X_{\text{seq}}) > 0$.

This assumption is natural in KT. Two response sequences may involve similar exercise and concept identities but induce different optimization effects when their local transitions or cumulative structural patterns differ.

PROPOSITION 1. *Under Assumption 1, we have $I(U_t; X_{\text{str}}) > I(U_t; X_{\text{seq}})$, $I(U_t; X_{\text{seq}}) \geq I(U_t; X_Q)$, and $I(U_t; X_{\text{seq}}) \geq I(U_t; X_C)$. Consequently, the structural representation X_{str} retains strictly more information about the one-step utility U_t than the direct sequence representation X_{seq} , while X_{seq} is at least as informative as the question-only and concept-only representations.*

PROOF. Since X_Q and X_C are deterministic projections of X_{seq} , the data processing inequality yields $I(U_t; X_{\text{seq}}) \geq I(U_t; X_Q)$ and $I(U_t; X_{\text{seq}}) \geq I(U_t; X_C)$. Likewise, X_{seq} is a deterministic projection of X_{str} , so $I(U_t; X_{\text{str}}) \geq I(U_t; X_{\text{seq}})$. Moreover, by the chain rule for mutual information:

$$I(U_t; X_{\text{str}}) = I(U_t; X_{\text{seq}}) + I(U_t; X_{\text{loc}}, X_{\text{cum}} \mid X_{\text{seq}}).$$

The conditional mutual information term is strictly positive by Assumption 1. Hence, $I(U_t; X_{\text{str}}) > I(U_t; X_{\text{seq}})$. Combining the above inequalities proves the proposition. $\square$

Proposition 1 is a representation-level result. It shows that, under Assumption 1, structural representations can preserve strictly more utility-relevant information than non-structural representations. It does not claim that every scalar metric derived from X_{str} is automatically optimal, but it does justify why metrics constructed from structural representations are better positioned to support curriculum decisions than question-only, concept-only, or identity-only sequence signals.

In our framework, the concrete structural signal is derived from the standardized dynamic Q -graph entropy and dynamic S -graph entropy, namely $\phi_{\text{str}}(R) = (H'(G_{DQ}(R)), H'(G_{DS}(R)))$. These quantities are computed from the dynamic Q -graph and dynamic S -graph, respectively, and therefore are measurable with respect to a structural representation. Proposition 1 explains why such a structural basis is theoretically preferable to non-structural alternatives for utility-aware sample discrimination.

7.1.2 Analysis on the RL-Based Scheduler. We next justify the RL-based scheduler from a sequential decision-making perspective. Curriculum scheduling is not a one-step ranking problem. A sampling decision affects not only the current optimization outcome, but also future training states and the usefulness of later samples. This naturally leads to an MDP formulation.

Let the curriculum scheduling process be modeled as the finite-horizon MDP in Section 5.2.2. To keep notation consistent, let the one-step reward be denoted by $r(s, a) = \mathbb{E}[L_{t-1} - L_t \mid S_t = s, A_t = a]$, that is, the expected validation-loss reduction induced by taking action a at state s . This is the action-conditioned form of the reward used in Equation (13). For any scheduler policy π , define the action-value function as:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=t}^T \gamma^{k-t} r(S_k, A_k) \mid S_t = s, A_t = a \right].$$

Let Q^* and V^* denote the optimal action-value and value functions. By the Bellman optimality equation, $Q^*(s, a) = r(s, a) + \gamma \mathbb{E}[V^*(S_{t+1}) \mid S_t = s, A_t = a]$. Hence, the optimal scheduler evaluates an action using both its immediate reward and its effect on future training states.

By contrast, a myopic heuristic scheduler chooses actions according to an immediate proxy $g(s, a)$, for example loss, uncertainty, or entropy, without optimizing the future-value term. Formally, $\pi_{\text{myopic}}(s) \in \arg \max_{a \in \mathcal{A}} g(s, a)$.

PROPOSITION 2. *Consider the curriculum scheduling MDP in Section 5.2.2. Suppose there exist a state $s_0 \in \mathcal{S}$ and two actions $a_1, a_2 \in \mathcal{A}$ such that $g(s_0, a_1) \geq g(s_0, a_2)$ but $Q^*(s_0, a_1) < Q^*(s_0, a_2)$.*

Then, any myopic heuristic scheduler that selects a_1 at s_0 is suboptimal at s_0 . In contrast, any optimal RL scheduler selects an action in $\arg \max_{a \in \mathcal{A}} Q^*(s_0, a)$ and therefore prefers a_2 to a_1 .

PROOF. Since $Q^*(s_0, a_1) < Q^*(s_0, a_2)$, action a_1 cannot be optimal at state s_0 . If a myopic heuristic scheduler selects a_1 at s_0 , then its value at s_0 is $V^{\pi_{\text{myopic}}}(s_0) = Q^{\pi_{\text{myopic}}}(s_0, a_1)$. For the fixed first action a_1 , the optimal continuation policy yields value $Q^*(s_0, a_1)$, which is at least as large as the value obtained by following any particular continuation policy after taking a_1 . Therefore, $Q^{\pi_{\text{myopic}}}(s_0, a_1) \leq Q^*(s_0, a_1)$. Combining this with the strict inequality in the proposition gives:

$$V^{\pi_{\text{myopic}}}(s_0) = Q^{\pi_{\text{myopic}}}(s_0, a_1) \leq Q^*(s_0, a_1) < Q^*(s_0, a_2) \leq V^*(s_0).$$

Hence, $V^{\pi_{\text{myopic}}}(s_0) < V^*(s_0)$, which proves that any myopic scheduler selecting a_1 at s_0 is suboptimal. On the other hand, an optimal RL scheduler acts by maximizing $Q^*(s, a)$ at each state, so it selects an action in $\arg \max_{a \in \mathcal{A}} Q^*(s_0, a)$ and therefore prefers a_2 over a_1 . $\square$

Proposition 2 clarifies the advantage of the RL-based scheduler. The difference is not merely algorithmic. A myopic heuristic ranks actions using an immediate proxy $g(s, a)$, whereas the RL scheduler ranks them using $Q^*(s, a)$, which explicitly combines current reward and future value. Therefore, whenever two actions have similar or even reversed proxy rankings but induce different future training states, a myopic heuristic may choose a suboptimal action, while the RL scheduler can still prefer the action with larger long-horizon utility.

This situation is natural in our setting. Repeatedly selecting the currently hardest or most uncertain samples may improve an immediate proxy, yet it may also push the model toward unstable regions, over-focus on noisy data, or neglect easier but informative samples that better prepare the model for later training. These long-horizon consequences are not captured by one-step heuristics, but they are captured by the RL objective through the value function.

7.2 Complexity Analysis

This section analyzes the theoretical complexity of the main components in R²GCuRL. We separate the discussion into graph construction, graph encoding, sequence modeling, and scheduling overhead, and compare the resulting complexity with three representative baselines: DyGKT [8], HD-KT [30], and PEBG [28]. Let N denote the number of response sequences, T the average sequence length, C the number of concepts associated with each response, d the hidden dimension, and Q the total number of exercises. Following common KT settings, we assume $Q > N \gg C \approx T \approx d$. This subsection focuses on training-time complexity; the practical runtime implications are discussed further in the efficiency analysis. Table 3 summarizes the complexity comparison between R²GCuRL and the baseline methods.

- PEBG constructs a bipartite graph between exercises and concepts with complexity $O(N \cdot T \cdot C)$. The joint optimization module introduces $O(d^2)$ parameters and complexity $O(Q^2 \cdot d^2)$. Its KT model (DKT) adds $O(d^2)$ parameters and $O(N \cdot T \cdot d^2)$ complexity. Overall, PEBG has $O(d^2)$ parameters and complexity $O(Q^2 \cdot d^2 + N \cdot T \cdot d^2)$.
- DyGKT employs a dynamic graph structure for response sequences. Its core components, mainly fully connected or attention-based layers, use $O(d^2)$ parameters. Processing each response step with its dynamic graph context results in complexity $O(N \cdot C \cdot T \cdot d^2)$. Hence, DyGKT has $O(d^2)$ parameters and complexity $O(N \cdot C \cdot T \cdot d^2)$.
- HD-KT comprises three modules. A knowledge state-guided anomaly detector with Bi-LSTM, VAE, and CNN components, each requiring $O(d^2)$ parameters and $O(N \cdot T \cdot d^2)$ complexity. A student profile-guided anomaly detector using attention over hidden states, costing $O(N \cdot T \cdot d^2)$.

Table 3. Complexity Analysis of the Main Methods

	#Params	Complexity
PEBG	$O(d^2)$	$O(Q^2 \cdot d^2 + N \cdot T \cdot d^2)$
DyGKT	$O(d^2)$	$O(N \cdot C \cdot T \cdot d^2)$
HD-KT	$O(d^2)$	$O(N \cdot T \cdot d^2)$
R ² GCurL	$O(d^2)$	$O(N \cdot T \cdot d^2)$

A KT component (LPKT) with $O(d^2)$ parameters and $O(N \cdot T \cdot d^2)$ complexity. Overall, HD-KT has $O(d^2)$ parameters and complexity $O(N \cdot T \cdot d^2)$.

For R²GCurL, dynamic graph construction for all sequences costs $O(N \cdot C \cdot T)$. The graph encoder introduces $O(d^2)$ parameters and contributes $O(N \cdot C \cdot T \cdot d)$ complexity, while the backbone KT model contributes $O(d^2)$ parameters and $O(N \cdot T \cdot d^2)$ complexity. The difficulty measurer is computed once before training, requiring $O(N \cdot C \cdot T)$ for graph-based statistics and $O(N \cdot \log N)$ for sorting. The scheduler adds an extra $O(N)$ selection overhead per update. The dominant training term therefore remains $O(N \cdot T \cdot d^2)$, while graph construction and scheduling act as auxiliary overheads. In other words, our method adds structural modeling and adaptive scheduling costs on top of the KT backbone, but it does not change the dominant asymptotic order of the underlying sequence model.

8 Experiments

In this section, we evaluate the performance of the R²GCurL framework on the KT task on five real-world datasets. We first introduce the datasets, the baseline models, and hyper-parameter settings. Then, we compare R²GCurL with baseline models and analyze the results. Additionally, we conduct ablation study, hyper-parameter analysis, efficiency study, scalability study, and convergence analysis for model training. Finally, we perform a knowledge state analysis to further understand the KT models.

8.1 Datasets

The proposed R²GCurL is evaluated on five widely used real-world KT datasets [3]. ASSISTments2009, ASSISTments2012, and ASSISTments2017 were collected from the ASSISTments platform in the corresponding school years. Ednet-KT1-Sub and Junyi-Sub are sampled subsets from large-scale tutoring platforms, each containing 5,000 students for our experiments. Dataset statistics are summarized in Table 4. To evaluate robustness, we additionally create synthetic noisy response sequences by randomly sampling exercises $q_i \in Q$ and responses $r_i \in \{0, 1\}$. We consider five noise levels (10%, 20%, 30%, 40%, and 50%) together with the original 0% setting. We treat these synthetic settings as a controlled approximation for studying robustness under unstable response patterns, rather than as a complete reproduction of all real-world noise sources.

8.2 Compared Methods and Experimental Setup

We compare our R²GCurL framework with 13 baselines, including four sequence-based KT models (DKT [41], DKVMN [63], SAKT [34], AKT [15]), four graph-based KT models (GKT [33], GIKT [60], PEBG [28], DyGKT [8]), one robust KT model (HD-KT [30]), and four CL methods (BabyStep [50], OnePass [53], CurGraph [55], RLCurL [20]). Since existing methods lack a difficulty measurer for KT tasks, our comparison emphasizes the training scheduler modules. To investigate whether

Table 4. Statistics and Properties of the Five KT Datasets

	ASSISTments			Ednet-	Junyi-
	2009	2012	2017	KT1-Sub	Sub
#Res. Seq.	4,151	29,018	1,709	5,000	5,000
#Exercises	16,891	53,091	3,162	11,896	674
#Concepts	110	265	102	1,755	39
#Res. Int.	325,637	2,711,813	942,816	571,758	481,525
Avg. Length	78	93	552	114	96
Max Length	1,261	2,089	3,057	11,326	6,788
Length Var.	24,293	23,392	175,529	243,326	104,028

“Res. Seq.” stands for “Response Sequence,” “Res. Int.” stands for “Response Interactions” which refers to the total number of times students have answered exercises, and “Length Var.” represents the variance in response sequence lengths.

the RL-based scheduler in our R^2GCuRL is truly worthy, we further compare it with four simpler alternatives that avoid RL-based policy learning and instead rely on lightweight heuristic scheduling strategies, including loss-based curriculum, hard example mining, uncertainty-based curriculum, and self-paced learning. To explore R^2GCuRL ’s flexibility, we propose four variants: R^2GCuRL -GNN, R^2GCuRL -DKVMN, R^2GCuRL -SAKT, and R^2GCuRL -AKT. R^2GCuRL -GNN employs GNN models for KT task, while the others integrate R^2GCuRL -GNN with corresponding sequence-based KT models (as detailed in Section 5.1.4).

The hyper-parameters for R^2GCuRL include a dropout rate of $p = 0.2$, learning rate of 0.0001, and hidden size $d = 256$. We use two GNN layers, the Adam optimizer, and the **Proximal Policy Optimization (PPO)** algorithm [45] for RL, with an MLP as the policy model. The hyperparameters of PPO follow the Stable-Baselines3 [44] defaults, with a learning rate of $3e-4$ and an exploration rate of 0. All benchmarks adopt the same class labels, feature vectors, and dataset splits (60% train, 20% validation, 20% test). Training continues until the validation AUC reaches its peak. Experiments are implemented using TensorFlow [1], PyTorch [37], PyTorch Geometric [14], OpenAI Gym [6], and Stable-Baselines3 [44], running on an NVIDIA A100-40GB GPU.

8.3 Performance Comparison

The experimental results for the KT task on the five datasets are illustrated in Table 5, showcasing the AUC metric for each model. Additional metrics, including RMSE and precision, are provided in Table 6. With the graph modeling module capturing the structural information within exercises and knowledge concepts, and the CL module guiding the learning process of KT models, the R^2GCuRL framework effectively enhances the robustness of KT.

We develop four variants of the R^2GCuRL framework and make comparisons with KT baseline models. As displayed in Table 5, three patterns can be observed: (1) The R^2GCuRL -GNN variant matches or surpasses sequence- and graph-based KT models, effectively leveraging structural information. The other three R^2GCuRL variants, when paired with DKVMN, SAKT, and AKT, show improvements across all datasets and noise levels. Specifically, the R^2GCuRL framework enhances DKVMN performance by an average of 6.85%, SAKT by 9.99%, and AKT by 2.79%. These enhancements underscore the robustness, flexibility, and scalability of the R^2GCuRL framework. (2) As the noisy ratio increases within the datasets, most baseline models exhibit varying degrees of performance degradation. Notably, the R^2GCuRL framework, particularly the R^2GCuRL -GNN variant, consistently demonstrates robustness across increasing noise levels compared to both

Table 5. Overall Comparison of AUC Performance across Five Real-World Datasets, Each Evaluated at Five Different Noise Levels

Dataset	Noisy Ratio	DKT	DKVMN	SAKT	AKT	GKT	GIKT	PEBG	DyGKT	HD-KT	R ² GCuRL (Ours)			
											GNN	DKVMN	SAKT	AKT
ASSISTments2009	0%	0.8209	0.8031	0.8077	0.8257	0.8136	0.8222	0.7215	0.5812	0.8185	0.8292	0.8292 _{↑2.35%}	0.8265 _{↑2.33%}	0.8331 _{↑0.90%}
	10%	0.8194	0.7809	0.8067	<u>0.8241</u>	0.8016	0.8211	0.6716	0.6102	0.8223	0.8190	0.8239 _{↑5.51%}	0.8152 _{↑1.05%}	0.8283 _{↑0.51%}
	20%	<u>0.8196</u>	0.7799	0.8049	0.8186	0.8110	0.8149	0.6774	0.6176	0.8122	0.8184	0.8127 _{↑4.21%}	0.8095 _{↑0.57%}	0.8203 _{↑0.21%}
	30%	0.8096	0.7810	0.8017	0.8128	0.8113	0.8117	0.6631	0.6377	0.8125	<u>0.8156</u>	0.8130 _{↑4.10%}	0.8054 _{↑0.46%}	0.8182 _{↑0.66%}
	40%	0.8090	0.7775	0.7933	0.8081	0.8095	0.8010	0.6568	0.6546	0.8119	0.8135	0.8121 _{↑4.45%}	0.7958 _{↑0.32%}	0.8099 _{↑0.22%}
	50%	0.8057	0.7757	0.7807	0.8059	0.8107	0.8050	0.6546	0.6532	<u>0.8108</u>	0.7991	0.8122 _{↑4.71%}	0.7860 _{↑0.68%}	0.8088 _{↑0.36%}
ASSISTments2012	0%	0.7280	0.7204	0.7135	0.7417	N/A	0.7306	0.7155	0.7136	0.7488	0.7485	0.7480 _{↑3.83%}	0.7484 _{↑4.89%}	0.7525 _{↑1.46%}
	10%	0.7152	0.7159	0.6518	0.7320	N/A	0.7311	0.6926	0.7162	0.7336	0.7332	0.7316 _{↑2.19%}	<u>0.7358</u> _{↑12.89%}	0.7364 _{↑0.60%}
	20%	0.7126	0.6741	0.6439	0.7241	N/A	0.7248	0.6792	0.7050	0.7280	0.7315	0.7276 _{↑7.94%}	<u>0.7328</u> _{↑13.81%}	0.7341 _{↑1.38%}
	30%	0.7111	0.6518	0.6430	0.7192	N/A	0.7106	0.6452	0.6908	0.7223	0.7350	0.7201 _{↑10.48%}	<u>0.7370</u> _{↑14.62%}	0.7383 _{↑2.66%}
	40%	0.7054	0.6250	0.6254	0.7075	N/A	0.6979	0.6407	0.6741	0.7229	0.7308	0.7175 _{↑14.8%}	0.7349 _{↑17.51%}	0.7347 _{↑3.84%}
	50%	0.7071	0.6152	0.5670	0.6984	N/A	0.7040	0.6351	0.6583	0.7238	0.7318	0.7114 _{↑15.64%}	<u>0.7351</u> _{↑29.65%}	0.7380 _{↑5.67%}
ASSISTments2017	0%	0.7101	0.7000	0.6493	0.7384	0.7110	0.7362	0.7159	0.7000	0.7116	0.7648	0.7687 _{↑9.81%}	0.7731 _{↑19.07%}	0.7824 _{↑5.96%}
	10%	0.7087	0.6784	0.6407	0.7213	0.7091	0.7309	0.7034	0.6888	0.7001	0.7517	0.7238 _{↑6.69%}	0.7416 _{↑15.75%}	0.7502 _{↑4.01%}
	20%	0.7056	0.6776	0.6326	0.7209	0.7057	0.7201	0.7022	0.6888	0.7158	<u>0.7465</u>	0.7233 _{↑6.74%}	0.7437 _{↑17.56%}	0.7536 _{↑4.54%}
	30%	0.7063	0.6796	0.6300	0.7185	0.6978	0.7109	0.6978	0.6711	0.7068	<u>0.7454</u>	0.7266 _{↑6.92%}	0.7420 _{↑17.88%}	0.7496 _{↑4.33%}
	40%	0.7062	0.6804	0.6305	0.7124	0.6946	0.7053	0.6919	0.6520	0.7204	0.7467	0.7231 _{↑6.28%}	0.7433 _{↑17.89%}	0.7338 _{↑3.0%}
	50%	0.7054	0.6822	0.6289	0.7059	0.6889	0.7035	0.6913	0.6375	0.7099	0.7521	0.7177 _{↑5.2%}	<u>0.7437</u> _{↑18.25%}	0.7076 _{↑0.24%}
Ednet-KT1-Sub	0%	0.6742	0.6748	0.6534	0.7169	N/A	0.7014	0.7020	0.6399	0.6795	0.7448	<u>0.7455</u> _{↑10.48%}	0.7403 _{↑13.30%}	0.7471 _{↑4.21%}
	10%	0.6745	0.6642	0.6516	0.6938	N/A	0.6891	0.6958	0.6401	0.6623	<u>0.7357</u>	0.7335 _{↑10.43%}	0.7344 _{↑12.71%}	0.7431 _{↑7.11%}
	20%	0.6744	0.6647	0.6515	0.6942	N/A	0.6933	0.6899	0.6397	0.6608	0.7174	0.7299 _{↑9.81%}	0.7341 _{↑12.68%}	0.7346 _{↑5.82%}
	30%	0.6731	0.6638	0.6520	0.6958	N/A	0.6862	0.6810	0.6339	0.6650	0.7248	0.7122 _{↑7.29%}	0.7191 _{↑10.29%}	0.7329 _{↑5.33%}
	40%	0.6732	0.6629	0.6514	0.6966	N/A	0.6988	0.6749	0.6246	0.6517	<u>0.7254</u>	0.7068 _{↑6.62%}	0.6946 _{↑6.63%}	0.7282 _{↑4.54%}
	50%	0.6736	0.6622	0.6510	0.6971	N/A	0.7060	0.6665	0.6175	0.6592	0.7002	0.7304 _{↑10.30%}	0.6932 _{↑6.48%}	<u>0.7129</u> _{↑2.27%}
Junyi-Sub	0%	0.7513	0.7514	0.7310	0.7801	0.7335	0.7639	0.7761	0.7609	0.7842	0.7856	0.7819 _{↑4.06%}	0.7813 _{↑6.88%}	0.7924 _{↑1.58%}
	10%	0.7500	0.7465	0.7306	0.7671	0.7396	0.7643	0.7764	0.7739	0.7724	0.7703	0.7785 _{↑4.29%}	0.7719 _{↑5.65%}	0.7863 _{↑2.5%}
	20%	0.7506	0.7422	0.7285	0.7649	0.6882	0.7689	0.7766	0.7786	<u>0.7832</u>	0.7703	0.7749 _{↑4.41%}	0.7698 _{↑5.67%}	0.7845 _{↑2.56%}
	30%	0.7513	0.7361	0.7273	0.7656	0.7319	0.7689	0.7757	0.7790	0.7777	0.7487	0.7718 _{↑4.85%}	0.7661 _{↑5.33%}	0.7841 _{↑2.42%}
	40%	0.7500	0.7341	0.7256	0.7657	0.7355	0.7599	0.7739	0.7736	<u>0.7747</u>	0.7714	0.7717 _{↑5.12%}	0.7647 _{↑5.39%}	0.7830 _{↑2.26%}
	50%	0.7500	0.7347	0.7255	0.7661	0.7397	0.7624	0.7716	0.7684	0.7689	0.7392	0.7642 _{↑4.02%}	0.7561 _{↑4.22%}	0.7817 _{↑2.04%}
Avg. Rank	-	7.93	10.62	11.70	5.70	10.68	6.60	8.38	10.02	5.57	3.72	3.82	4.77	1.50

Results are highlighted where the best performance is in bold and the second-best in underlined. The improvements of the R²GCuRL framework variants compared to their counterparts are calculated, with an improvement denoted by an up arrow ↑, and a negative improvement by a down arrow ↓. “N/A” indicates experiments that were unable to complete within the experimental time limit. “Avg. Rank” represents the average ranking of the models across all datasets.

Table 6. Performance Comparison of RMSE and Precision on Two Datasets (50% Noise)

Dataset	ASSISTments2017		Junyi-Sub	
	Metric	RMSE ↓ Precision ↑	RMSE ↓ Precision ↑	
DKT	0.4736	0.5827	0.3642	0.8119
DKVMN	0.4825	0.5720	0.3743	0.8087
SAKT	0.5036	0.5038	0.3826	0.8054
AKT	0.4834	0.5772	0.3697	0.8159
GKT	0.4848	0.5553	0.3697	0.8131
GIKT	0.4839	0.4928	0.3591	0.8053
PEBG	0.4787	0.5479	0.3654	0.8029
DyGKT	0.4805	0.5879	0.4073	0.8333
HD-KT	0.4830	0.6041	0.3567	0.8401
R ² GCuRL-GNN	0.4372	<u>0.6116</u>	0.3704	0.8375
R ² GCuRL-DKVMN	0.4510	0.5787	0.3476	0.8413
R ² GCuRL-SAKT	<u>0.4499</u>	0.6529	<u>0.3471</u>	<u>0.8470</u>
R ² GCuRL-AKT	0.4657	0.5229	0.3456	0.8513

sequence-based and graph-based models. For instance, while traditional models like SAKT and PEBG show marked performance drops on the ASSISTments2012 dataset, R²GCuRL variants manage to maintain closer results to their peak performance. (3) The R²GCuRL framework achieves the best

Table 7. Ablation Study and AUC Comparison of Curriculum Learning Baselines on SAKT-GNN, an Enhanced SAKT Model with Our Dynamic Graph Modeling Module

Models		ASSISTments2012						Ednet-KT1-Sub					
		0%	10%	20%	30%	40%	50%	0%	10%	20%	30%	40%	50%
Baselines on Scheduler	OnePass [53]	0.7184	0.7007	0.6847	0.6890	0.7057	0.7076	0.7162	0.7210	0.7130	0.6959	0.6658	0.6637
	BabyStep [50]	0.7178	0.6951	0.6977	0.6741	0.6782	0.6959	0.7140	0.7217	0.7198	0.7012	0.6800	0.6675
	CurGraph [55]	0.7214	0.7060	0.6960	0.6902	0.7073	0.7108	0.7202	0.7186	0.7171	0.6968	0.6864	0.6572
	RLCurl [20]	0.7266	0.7154	0.7105	0.7135	0.7181	0.7280	0.7235	0.7224	0.7222	0.7045	0.6846	0.6754
Ablation Study	SAKT	0.7135	0.6518	0.6439	0.6430	0.6254	0.5670	0.6534	0.6516	0.6515	0.6520	0.6514	0.6510
	w/o GNN	0.7271	0.6312	0.6467	0.6475	0.6468	0.6302	0.6731	0.6555	0.6688	0.6679	0.6698	0.6884
	w/o Training Scheduler	0.7206	0.7256	0.6833	0.6594	0.6348	0.6309	0.7211	0.7244	0.7115	0.6880	0.6683	0.6558
	w/o Difficulty Measurer	0.7192	0.7273	0.6966	0.6754	0.6790	0.6963	0.7182	0.7287	0.7163	0.6911	0.6977	0.6654
R²GCurl-SAKT (Ours)		0.7484	0.7358	0.7328	0.7370	0.7349	0.7351	0.7403	0.7344	0.7341	0.7191	0.6946	0.6932

Bold values indicate the best performance.

results on the five datasets, and the overall performance is 4.16% higher than the best SOTA method (HD-KT) on average. This superior performance is primarily due to the R²GCurl framework's ability to effectively capture structural information and adaptively manage the learning process of KT models, demonstrating its robustness and effectiveness in diverse learning environments.

Due to the lack of CL methods in the KT field, we implement four training scheduler baselines to validate the performance of R²GCurl. For a fair comparison, we employ the same difficulty measurer and integrate the GNN-enhanced SAKT model into these CL baseline methods as in our R²GCurl framework. The comparison in Table 7 clearly illustrates the superiority of the R²GCurl-SAKT variant over existing CL baselines across different noise levels and datasets. In particular, compared with the strongest non-RL CL baselines (i.e., OnePass, BabyStep, and CurGraph), R²GCurl-SAKT achieves average absolute AUC gains of 3.18% on ASSISTments2012 dataset. The pre-defined CL methods (OnePass [53], BabyStep [50], and CurGraph [55]) show inferior performance compared to the automatic CL method (RLCurl [20]) across various experimental conditions, highlighting the effectiveness of adaptive scheduling of training samples. Specifically, our R²GCurl-SAKT consistently outperforms all CL baselines, demonstrating both higher model performance and remarkable stability under increasing noise conditions. This consistency underscores the robustness of the R²GCurl framework, showcasing its effectiveness in handling the challenges posed by noisy data in KT tasks.

8.4 Ablation Study

The ablation study results clearly show the positive contribution of each component to the overall performance of the R²GCurl-SAKT framework in Table 7. Several key observations can be drawn from the results of the ablation study:

- The R²GCurl-SAKT (Ours) model consistently achieves the highest AUC values across all noisy ratios (0% to 50%) on both the ASSISTments2012 and Ednet-KT1-Sub datasets. This demonstrates that the integration of all components provides the best results.
- The model without GNN module (w/o GNN) shows significantly lower performance compared to the full model. This is especially noticeable on the Ednet-KT1-Sub dataset. This suggests that the dynamic graph modeling module is crucial for capturing the relationships within the data.
- The performance of the model without the training scheduler (w/o Training Scheduler) drops notably as the noisy ratio increases. While it performs reasonably well with a small amount of data, its performance degrades significantly as more data is included. This indicates that

the training scheduler is vital for maintaining model stability and performance when dealing with varying data proportions.

- Similar to the training scheduler, removing the difficulty measurer (w/o Difficulty Measurer) also leads to a decline in performance as the noisy ratio grows. In this specific experiment, the difficulty measurer is replaced by assigning each sample a random value. This shows that the derived structure-aware difficulty signal plays a key role in distinguishing different training conditions induced by response sequences and in adapting the curriculum accordingly, which is essential for the model’s overall generalization ability.

In summary, the ablation study effectively validates that each component contributes positively and significantly to the superior performance of the R²GCurL model.

8.5 Hyper-Parameter Analysis

We further examine the RL-based scheduler in terms of hyper-parameter sensitivity, action-space design, and algorithmic choice. This analysis studies the stability of the RL scheduler and the conditions under which its adaptive behavior is most beneficial. In our experiments, simpler heuristic schedulers remain competitive in relatively clean settings, while the RL scheduler becomes more useful under noisier and more heterogeneous conditions, where fixed curricula are less reliable.

8.5.1 Learning Rate (*lr*) and Exploration Rate (*er*). Figure 7(a) shows that the RL scheduler is robust to both the policy learning rate and the exploration rate. In general, smaller learning rates together with mild exploration lead to more reliable performance. This trend suggests that the scheduler benefits from gradual policy updates, which reduce abrupt changes in sample selection during training. By contrast, overly aggressive exploration tends to introduce large shifts in the selected difficulty range, making the curriculum less consistent from one stage to the next.

8.5.2 Action Space Design. Figure 7(b) compares discrete and continuous action spaces for the RL scheduler. The discrete action space consistently performs better across the evaluated noise levels. A likely reason is that curriculum updates are inherently stage-wise, where selecting among a small set of difficulty adjustments leads to smoother transitions in the training data distribution. In contrast, a continuous action space allows much larger variation in the curriculum parameters ($\Delta\mu \in [0, 1]$ and $\Delta\sigma^2 \in [0.01, 0.1]$), which makes the scheduler more prone to unstable updates. This result suggests that, for curriculum control, a constrained and stable action design is more effective than a highly flexible one.

8.5.3 RL Algorithm. Figure 7(c) compares the performance of various RL algorithms for our RL module. We find that PPO [45] consistently outperforms other algorithms like A3C [31], DDPG [25], and DQN [32]. PPO’s strength lies in its stability and sample efficiency. PPO utilizes a clipped surrogate objective function, which prevents overly large policy updates. This mechanism ensures that the training remains stable, even in the presence of noisy data, which is a common challenge in educational datasets. Other algorithms, such as DQN, are less stable, while DDPG and A3C, though generally robust, do not match the performance of PPO in this specific CL setting. The stability provided by PPO allows the model to reliably find and follow an optimal curriculum, leading to superior KT performance.

8.5.4 Weight of the Difficulty Score. Figure 6 shows that balancing the standardized dynamic Q-graph entropy $H'(G_D^i Q)$ and dynamic S-graph entropy $H'(G_D^i S)$ with an appropriate λ is crucial for optimal performance. The best or near-best results of both R2GCurL-GNN and R2GCurL-SAKT are generally obtained around $\lambda = 0.5$, suggesting that local step-wise structural continuity and cumulative historical structural continuity provide complementary signals. When λ is too small,

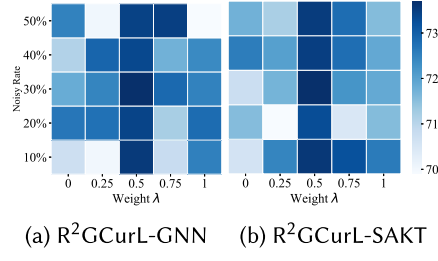


Fig. 6. Hyper-parameter analysis on the ASSISTments2012. The hyper-parameter λ controls the weight of the difficulty score in the difficulty measurer defined in Section 5.2.1.

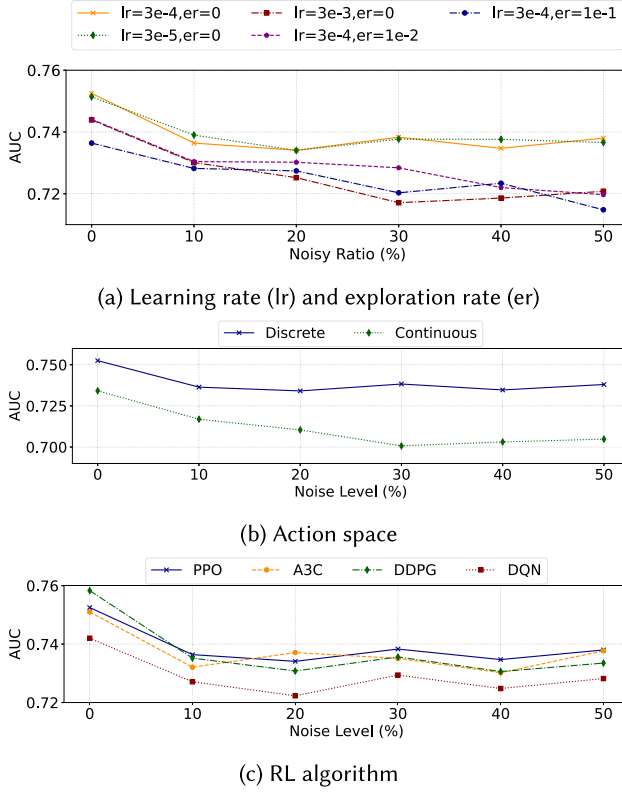


Fig. 7. Hyper-parameter analysis of the RL module on ASSISTments2012 under different noisy ratios.

the difficulty signal mainly depends on local Q-graph transitions and may overlook accumulated learning patterns. When λ is too large, it mainly depends on the cumulative S-graph and may become less sensitive to local exercise-concept changes. Therefore, a balanced λ yields a more effective structure-aware curriculum signal.

8.6 Effectiveness and Necessity of the RL-Based Scheduler

To examine whether the RL-based scheduler is truly necessary beyond simpler heuristic alternatives, we compare R²GCurL with four non-RL curriculum strategies under the same backbone and training protocol, including loss-based sampling, hard example mining, uncertainty-based scheduling, and

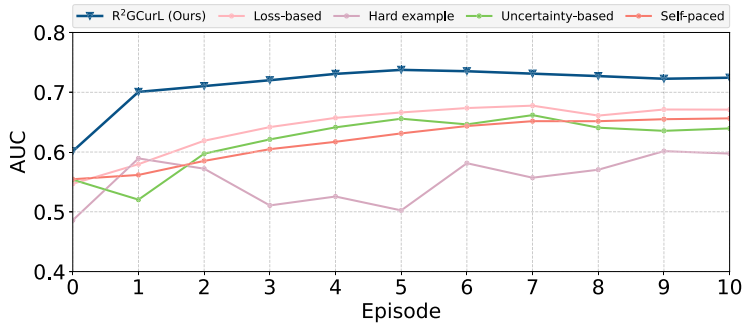


Fig. 8. Ablation analysis on the RL-based scheduler on ASSIST2017 (50% noise) with valid AUC.

Table 8. Different Curriculum Strategies on ASSIST2017 with 50% Noise

Method	AUC	Total runtime (min)	Scoring overhead (min)	GPU (MB)
w/o CL	0.6593	60.3	N/A	3,142
Loss-based Sampling	0.6913	101.3	37.7	3,253
Hard Example Mining	0.6579	109.6	38.0	3,879
Self-paced Learning	0.6864	95.6	38.9	3,313
Uncertainty-based Scheduling	0.6601	94.5	41.1	3,822
R ² GCurL (Ours)	0.7437	114.8	48.1	3,894

All methods are evaluated under the same experimental settings. We report the total wall-clock runtime together with the scoring overhead, where the latter is included in the total runtime. The scoring overhead specifically refers to the computation of the difficulty scores.

self-paced learning. Figure 8 presents the validation AUC trajectories during training, and Table 8 reports the final AUC together with runtime and memory statistics. Several observations can be made. First, R²GCurL consistently achieves the strongest performance throughout training and converges to the best final result among all compared strategies. Moreover, its advantage is not limited to the final epoch. As shown in Figure 8, the validation AUC of R²GCurL rises rapidly in the early stage and remains stably above all heuristic baselines across the entire training process, indicating a more effective and robust sample-selection behavior rather than an incidental improvement at convergence. By contrast, hard example mining exhibits larger fluctuations and persistently weaker performance, suggesting that aggressively emphasizing difficult instances is insufficient and may even destabilize optimization under heavy noise. Loss-based, uncertainty-based, and self-paced strategies are comparatively more stable, but they still saturate at clearly lower performance levels, implying that fixed heuristic rules cannot adequately adapt sample selection to the evolving training state. In addition, although the RL-based scheduler introduces extra runtime and scoring overhead, Table 8 shows that this additional cost remains moderate and does not alter the overall efficiency regime of the method. Taken together, these results suggest that the benefit of the RL-based scheduler does not come merely from adding complexity, but from its ability to adaptively coordinate the curriculum throughout training, which cannot be fully matched by fixed loss-based, uncertainty-based, or easy-to-hard heuristics.

Table 9. Efficiency Analysis on ASSISTments2012 (50% Noise)

	AUC	Training Phase			Inference Phase	
		Model Size (MB)	GPU (MB)	Training (min)	GPU (MB)	ms/sample
DKT	0.7071	2.79	1,302	62	156	0.82
DKVMN	0.6152	1.88	2,478	186	669	1.40
SAKT	0.5670	4.00	1,280	109	147	0.72
AKT	0.6984	17.56	1,807	113	393	1.63
HD-KT	0.7238	25.58	3,944	106	420	1.84
DyGKT	0.6583	0.29	3,330	99	143	0.84
GIKT	0.7040	20.35	3,097	144	1,144	9.25
PEBG	0.6351	105.91	2,828	171	2,001	0.87
R ² GCuRL-GNN	0.7318	7.36	1,905	87	1,975	1.87
R ² GCuRL-DKVMN	0.7114	9.23	4,253	197	1,983	2.83
R ² GCuRL-SAKT	0.7351	11.73	3,113	125	1,985	2.11
R ² GCuRL-AKT	0.7380	25.37	3,540	136	2,218	2.67

Training-phase and inference-phase GPU memory is measured as peak reserved GPU memory with batch size 32 and sequence length 200. Total training time reports the time to convergence under the same early stopping criterion, rather than the cost of running a fixed number of epochs.

8.7 Efficiency, Scalability, and Convergence Analysis

To further validate the efficiency, scalability, and convergence of our R²GCuRL framework, we conduct comparisons with baseline models under the same experimental conditions.

8.7.1 Efficiency Analysis. Table 9 reports the efficiency comparison on ASSISTments2012 with 50% noise, including model size, peak GPU memory usage in both training and inference phases, total training time to convergence, and inference latency in ms/sample. These results provide a more complete view of the computational characteristics of graph-based KT methods beyond predictive accuracy alone. In general, graph-based models incur higher computational cost than lightweight sequential baselines, especially in GPU memory usage, due to the additional dynamic graph construction and message-passing operations. Nevertheless, the proposed R²GCuRL framework adds moderate overhead relative to the backbone. For example, although R²GCuRL variants require more GPU memory than their parent backbones, their inference latency remains within 1.87–2.83 ms/sample, which is substantially lower than that of GIKT (9.25 ms/sample) and comparable to several other graph-based baselines. This suggests that the additional cost introduced by dynamic graph modeling does not lead to prohibitive online serving latency. Moreover, the total training time in Table 9 is measured as time to convergence under the same early stopping criterion. Under this setting, R²GCuRL-GNN reaches convergence in 87 minutes, while R²GCuRL-AKT attains the best AUC with moderate additional cost relative to its backbone. Overall, these results indicate that the proposed graph-based design increases computation in a controlled manner and yields consistent performance gains without introducing impractical deployment overhead.

8.7.2 Scalability Analysis. Figure 9 illustrates the scalability of different models as the size of the training dataset is adjusted to represent 25%, 50%, 75%, and 100% of the total data on both ASSISTments2012 and ASSISTments2017 datasets. The results indicate that as the training dataset grows, the R²GCuRL models consistently achieve the highest AUC, exhibiting a smoother and more stable upward trend compared to other models. This highlights its superior scalability and ability to effectively utilize larger datasets.

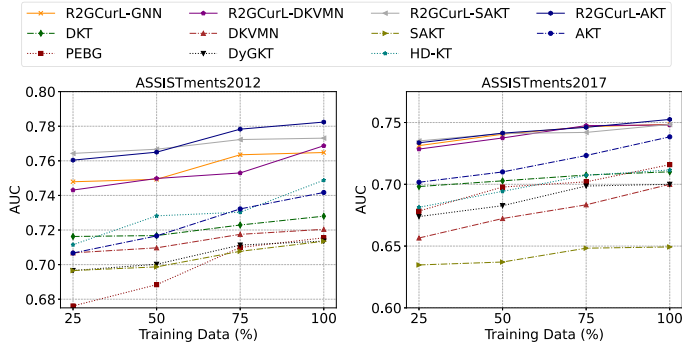


Fig. 9. Scalability Analysis on Two Datasets.

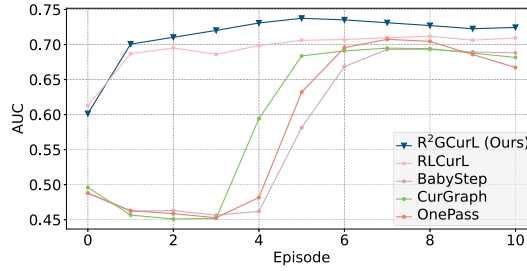


Fig. 10. Convergence analysis on ASSIST2017 (50% noise) with valid AUC among RL scheduler.

8.7.3 Convergence Analysis. As shown in Figure 10, which presents the convergence of models on the ASSISTments2017 dataset with a 50% noise ratio, our R²GCuRL framework not only converges faster but also reaches a higher and more stable performance level compared to other models. The R²GCuRL model achieves its peak performance in fewer training episodes than other CL models such as BabyStep and CurGraph. This rapid convergence demonstrates that our RL-based curriculum strategy effectively guides the model to learn efficiently, allowing it to quickly identify the optimal learning path and stabilize at a high AUC.

In summary, the R²GCuRL framework integrates dynamic graph-based modeling with reinforced CL to manage the learning process both adaptively and efficiently, resulting in enhanced performance as well as improved scalability and efficiency.

8.8 Dynamic Graph Entropy Analysis

Dynamic graph entropy is introduced because raw response sequences do not explicitly encode how question-concept structure evolves over time, whereas the dynamic Q -graph and dynamic S -graph make such structure explicit at local and historical levels. Under the current definition, dynamic graph entropy measures graph-transition continuity rather than uncertainty in the usual sense. The curriculum module therefore does not equate entropy itself with difficulty or noise. Instead, after sequence-level aggregation and standardization, it derives a structure-aware difficulty signal from these continuity-related quantities. This distinction is important because a clean topic switch and noisy corruption can both reduce continuity, but they do not induce the same structural pattern. A topic switch typically produces a coherent transition into a new concept region, whereas noisy responses tend to create more irregular and less consistent disruptions relative to both nearby interactions and the accumulated learning history. Accordingly, the analysis in Table 10 is intended

Table 10. Sequence-Level Entropy Analysis on ASSIST2017 under the 50% Noise Setting

Group	N	$H'(\mathcal{G}_{DQ}^i)$			$H'(\mathcal{G}_{DS}^i)$			Difficulty Signal		
		Mean	Std	Median	Mean	Std	Median	Mean	Std	Median
Clean sequences	3,298	0.6479	0.4261	0.7292	0.0670	0.9183	0.2899	-0.3574	0.5544	-0.4723
Noisy sequences	1,579	-1.3532	0.0424	-1.3550	-0.1400	1.1394	0.1892	0.7466	0.5718	0.5859

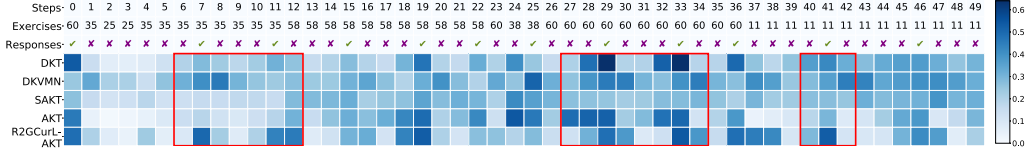


Fig. 11. Visualization of knowledge states derived from the KT models on the ASSISTments2017 dataset. The heatmap colors reflect the models' predicted correctness probability at each step, with darker colors indicating higher likelihoods.

to examine whether the continuity-based signal can distinguish these qualitatively different cases, rather than to claim that it fully characterizes all forms of educational difficulty or noise. The results show that noisy sequences shift more consistently toward the harder direction of the derived signal, while topic-switching behavior exhibits a separate pattern instead of collapsing into the same category as noise. This supports dynamic graph entropy as a more defensible basis for curriculum scoring than directly equating large structural change with noisy behavior.

8.9 Knowledge State Analysis

We evaluate interpretability at the trajectory level by examining whether predicted knowledge-state evolution remains locally stable and whether adjacent updates are aligned with observed concept and response changes.

We visualize the predicted correctness probabilities of different models in Figure 11, where darker colors indicate higher predicted values. The heatmap provides a qualitative view of how each model traces a student's knowledge-state trajectory over time. Compared with the baselines, R²GCurl-AKT shows smoother and more consistent predictions in the highlighted regions, while remaining well aligned with the observed interaction pattern. In the first red box (steps 6–12), after an incorrect response at step 6, DKT fluctuates more strongly, whereas R²GCurl-AKT changes more conservatively. In the second red box (steps 27–34), where correct and incorrect responses alternate, R²GCurl-AKT remains comparatively stable while still responding to the sequence. In the third red box (steps 40–42), several baselines exhibit sharp fluctuations, whereas R²GCurl-AKT maintains a smoother trajectory. Overall, Figure 11 suggests that our R²GCurl produces a less erratic and more stable knowledge-state trajectory under noisy interactions.

To complement the qualitative heatmap, we quantify trajectory-level behavior using two local proxies that directly reflect the interpretability criterion discussed above. First, to measure local stability under repeated practice on the same concept, we define the **Same-Concept Run Delta (SCRD)** as the average adjacent prediction change within maximal consecutive same-concept runs: $SCRD = \frac{1}{\sum_{s \in \mathcal{S}} (|s|-1)} \sum_{s=[a,b] \in \mathcal{S}} \sum_{t=a+1}^b |\hat{y}_t - \hat{y}_{t-1}|$, where $\mathcal{S}$ denotes the set of same-concept runs with length at least two. Lower SCRD indicates a smoother local trajectory when the concept context remains unchanged. Second, to measure whether local updates are aligned with the observed response signal, we define the **Response-Consistent Transition Rate (RCTR)**. Let $\Delta_t = \hat{y}_{t+1} - \hat{y}_t$ and let $r_t \in \{0, 1\}$ be the response at step t ; with a tolerance $\epsilon = 0.01$, a transition is counted as

Table 11. Trajectory-Level Interpretability Proxies on ASSIST2017 under the 50% Noise Setting for Selected Baselines and the Four R²GCurL Variants

Metric	Baselines				R ² GCurL Variants			
	DKT	DKVMN	SAKT	AKT	GNN	DKVMN	SAKT	AKT
SCRD ↓	0.097	0.160	0.176	0.134	0.098	0.078	0.066	0.125
RCTR ↑	0.447	0.493	0.467	0.474	0.446	0.592	0.506	0.479

consistent if $(r_t = 1 \wedge \Delta_t \geq -\epsilon)$ or $(r_t = 0 \wedge \Delta_t \leq \epsilon)$, and $RCTR = \frac{1}{T-1} \sum_{t=1}^{T-1} I_t$. Higher RCTR indicates that adjacent prediction updates move in a direction more consistent with whether the student answered correctly or incorrectly. We use these two metrics as compact trajectory-level diagnostics rather than as a complete or universal notion of interpretability. Table 11 shows that the R²GCurL variants maintain stable and response-aligned knowledge-state trajectories under noisy conditions. In particular, R²GCurL-DKVMN and R²GCurL-SAKT achieve both lower SCRD and higher RCTR than their corresponding backbones, indicating smoother same-concept evolution together with more response-consistent local updates. The R²GCurL-GNN and R²GCurL-AKT variants also remain competitive on both metrics, showing that the proposed design preserves favorable local trajectory characteristics while improving robustness. Overall, these results provide supportive evidence that R²GCurL can yield more coherent trajectory-level behavior beyond final predictive accuracy.

9 Conclusions and Future Work

R²GCurL unifies dynamic graph modeling and adaptive CL to improve personalized structural modeling, robustness to noisy interactions, and training efficiency in KT. By representing student trajectories as dynamic exercise–concept graphs, the framework captures both local interaction structure and cumulative historical structure. The curriculum module further derives a continuity-based structural signal from dynamic graph entropy and uses an RL-based scheduler to adapt curriculum emphasis according to model feedback. Across five real-world datasets and multiple noise settings, the proposed method consistently improves predictive performance while keeping computational overhead within a practical range. The empirical results further show that the RL-based scheduler provides clear advantages over simpler heuristic alternatives in noisy settings, and that the resulting knowledge-state trajectories are more coherent and response-aligned. In future work, we plan to validate these mechanisms in broader tutoring scenarios, extend the framework to multimodal educational data, and investigate lighter-weight adaptive schedulers together with richer trajectory-level interpretability analyses.

References

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. TensorFlow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, 265–283.
- [2] Ghodai Abdelrahman and Qing Wang. 2022. Deep graph memory networks for forgetting-robust knowledge tracing. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2022), 7844–7855.
- [3] Ghodai Abdelrahman, Qing Wang, and Bernardo Nunes. 2023. Knowledge tracing: A survey. *ACM Computing Surveys* 55, 11 (2023), 1–37.
- [4] Joseph E. Beck and June Sison. 2006. Using knowledge tracing in a noisy environment to measure student reading proficiencies. *International Journal of Artificial Intelligence in Education* 16, 2 (2006), 129–143.
- [5] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, 41–48.

- [6] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. OpenAI Gym. arXiv:1606.01540. Retrieved from <https://arxiv.org/abs/1606.01540>
- [7] Hao Cen, Kenneth Koedinger, and Brian Junker. 2006. Learning factors analysis—a general method for cognitive model evaluation and improvement. In *Proceedings of the International Conference on Intelligent Tutoring Systems*. Springer, 164–175.
- [8] Ke Cheng, Linzhi Peng, Pengyang Wang, Junchen Ye, Leilei Sun, and Bowen Du. 2024. DyGKT: Dynamic graph learning for knowledge tracing. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 409–420.
- [9] Albert T. Corbett and John R. Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modelling and User-Adapted Interaction* 4 (1994), 253–278.
- [10] Chaoran Cui, Yumo Yao, Chunyun Zhang, Hebo Ma, Yuling Ma, Zhaochun Ren, Chen Zhang, and James Ko. 2024. DGEKT: A dual graph ensemble learning method for knowledge tracing. *ACM Transactions on Information Systems* 42, 3 (2024), 78:1–78:24.
- [11] Jiajun Cui, Zeyuan Chen, Aimin Zhou, Jianyong Wang, and Wei Zhang. 2023. Fine-grained interaction modeling with multi-relational transformer for knowledge tracing. *ACM Transactions on Information Systems* 41, 4 (2023), 104:1–104:26.
- [12] Jiajun Cui, Minghe Yu, Bo Jiang, Aimin Zhou, Jianyong Wang, and Wei Zhang. 2024. Interpretable knowledge tracing via response influence-based counterfactual reasoning. In *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1103–1116.
- [13] Ziquan Fang, Yuntao Du, Xinjun Zhu, Danlei Hu, Lu Chen, Yunjun Gao, and Christian S. Jensen. 2022. Spatio-temporal trajectory similarity learning in road networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 347–356.
- [14] Matthias Fey and Jan E. Lenssen. 2019. Fast graph representation learning with PyTorch geometric. In *Proceedings of the ICLR Workshop on Representation Learning on Graphs and Manifolds*, 1–12.
- [15] Aritra Ghosh, Neil Heffernan, and Andrew S. Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2330–2339.
- [16] Tieliang Gong, Qian Zhao, Deyu Meng, and Zongben Xu. 2015. Why curriculum learning & self-paced learning work in big/noisy data: A theoretical perspective. *Big Data and Information Analytics* 1, 1 (2015), 111–127.
- [17] Teng Guo, Yu Qin, Yubin Xia, Mingliang Hou, Zitao Liu, Feng Xia, and Weiqi Luo. 2025. Enhancing knowledge tracing through decoupling cognitive pattern from Error-Prone data. In *Proceedings of the ACM on Web Conference 2025 (WWW '25)*. Guodong Long, Michale Blumstein, Yi Chang, Liane Lewin-Eytan, Zi Helen Huang, and Elad Yom-Tov (Eds.), ACM, 5108–5116.
- [18] Liangliang He, Xiao Li, Pancheng Wang, Jintao Tang, and Ting Wang. 2024. MAN: Memory-augmented attentive networks for deep learning-based knowledge tracing. *ACM Transactions on Information Systems* 42, 1 (2024), 8:1–8:22.
- [19] Shuyan Huang, Zitao Liu, Xiangyu Zhao, Weiqi Luo, and Jian Weng. 2023. Towards robust knowledge tracing models via k-sparse attention. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2441–2445.
- [20] Gaurav Kumar, George Foster, Colin Cherry, and Maxim Krikun. 2019. Reinforcement learning based curriculum optimization for neural machine translation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Vol. 1, 2054–2061.
- [21] Haoyang Li, Xin Wang, and Wenwu Zhu. 2023. Curriculum graph machine learning: A survey. In *Proceedings of the 32nd International Joint Conference on Artificial Intelligence*, 667–6682.
- [22] Yuchen Li, Rui Kong, Xinran Chen, Chengzhe Zhang, Jiamin Chen, Cheng Deng, Xinyu Ma, Haojie Zhang, Tianhao Peng, Hengyi Cai, et al. 2026. Probe-and-fetch: Dynamic KV cache pruning for accelerated long-context inference in web-scale AI search. In *Proceedings of the ACM Web Conference 2026*, 8127–8137.
- [23] Yuchen Li, Zhonghao Lyu, Yongqi Zhang, Hao Zhang, Tianhao Peng, Haoyi Xiong, Shuaiqiang Wang, Linghe Kong, Guihai Chen, and Dawei Yin. 2025. S³PRank: Towards satisfaction-oriented learning to rank with semi-supervised pre-training. *IEEE Transactions on Knowledge and Data Engineering* 38 (2025), 559–572.
- [24] Yuchen Li, Haoyi Xiong, Yongqi Zhang, Jiang Bian, Tianhao Peng, Xuhong Li, Shuaiqiang Wang, Linghe Kong, and Dawei Yin. 2025. RankElectra: Semi-supervised pre-training of learning-to-rank electra for web-scale search. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, 2415–2425.
- [25] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. arXiv:1509.02971. Retrieved from <https://arxiv.org/abs/1509.02971>
- [26] Jianhua Lin. 1991. Divergence measures based on the Shannon entropy. *IEEE Transactions on Information Theory* 37, 1 (1991), 145–151.

- [27] Fei Liu, Chenyang Bu, Haotian Zhang, Le Wu, Kui Yu, and Xuegang Hu. 2024. FDKT: Towards an interpretable deep knowledge tracing via fuzzy reasoning. *ACM Transactions on Information Systems* 42, 5 (2024), 139:1–139:26.
- [28] Yunfei Liu, Yang Yang, Xianyu Chen, Jian Shen, Haifeng Zhang, and Yong Yu. 2021. Improving knowledge tracing via pre-training question embeddings. In *Proceedings of the 29th International Conference on International Joint Conferences on Artificial Intelligence*, 1556–1562.
- [29] Frederic M. Lord. 2012. *Applications of Item Response Theory to Practical Testing Problems*. Routledge.
- [30] Haiping Ma, Yong Yang, Chuan Qin, Xiaoshan Yu, Shangshang Yang, Xingyi Zhang, and Hengshu Zhu. 2024. HD-KT: Advancing robust knowledge tracing via anomalous learning interaction detection. In *Proceedings of the ACM on Web Conference 2024*, 4479–4488.
- [31] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous methods for deep reinforcement learning. In *Proceedings of the International Conference on Machine Learning*. PMLR, 1928–1937.
- [32] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing Atari with deep reinforcement learning. arXiv:1312.5602. Retrieved from <https://arxiv.org/abs/1312.5602>
- [33] Hiromi Nakagawa, Yusuke Iwasawa, and Yutaka Matsuo. 2019. Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In *Proceedings of the IEEE/WIC/ACM International Conference on Web Intelligence*, 156–163.
- [34] Shalini Pandey and George Karypis. 2019. A self-attentive model for knowledge tracing. In *Proceedings of the 12th International Conference on Educational Data Mining (EDM '19)*. International Educational Data Mining Society, 384–389.
- [35] Shalini Pandey and Jaideep Srivastava. 2020. RKT: Relation-aware self-attention for knowledge tracing. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, 1205–1214.
- [36] Zachary A. Pardos and Neil T. Heffernan. 2010. Modeling individualization in a Bayesian networks implementation of knowledge tracing. In *Proceedings of the 18th International Conference on User Modeling, Adaptation, and Personalization (UMAP '10)*. Springer, 255–266.
- [37] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. 2017. Automatic differentiation in pytorch. In *Proceedings of the NeurIPS*.
- [38] Tianhao Peng, Xuhong Li, Haitao Yuan, Yuchen Li, and Haoyi Xiong. 2025. SOLA-GCL: Subgraph-oriented learnable augmentation method for graph contrastive learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, 19875–19883.
- [39] Tianhao Peng, Wenjun Wu, Haitao Yuan, Zhifeng Bao, Zhao Pengrui, Xin Yu, Xuetao Lin, Yu Liang, and Yanjun Pu. 2023. GraphRARE: Reinforcement learning enhanced graph neural network with relative entropy. In *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering (ICDE)*.
- [40] Tianhao Peng, Haitao Yuan, Yongqi Zhang, Yuchen Li, Peihong Dai, Qunbo Wang, Senzhang Wang, and Wenjun Wu. 2025. TagRec: Temporal-Aware graph contrastive learning with theoretical augmentation for sequential recommendation. *IEEE Transactions on Knowledge and Data Engineering* 37 (2025), 3015–3029.
- [41] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J. Guibas, and Jascha Sohl-Dickstein. 2015. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, Vol. 28, 505–513.
- [42] Yanjun Pu, Fang Liu, Rongye Shi, Haitao Yuan, Ruibo Chen, Tianhao Peng, and Wenjun Wu. 2024. ELAKT: Enhancing locality for attentive knowledge tracing. *ACM Transactions on Information System* 42, 4 (2024), 112:1–112:27.
- [43] Yang Qin, Xinning Zhu, Xiaosheng Tang, Chunhong Zhang, Kunbao Wu, Fengjie Chang, Jianzhou Diao, and Zheng Hu. 2025. Interpretable knowledge tracing with difficulty-aware attention and selective state space model. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Nicola Ferro, Maria Maistro, Gabriella Pasi, Omar Alonso, Andrew Trotman, and Suzan Verberne (Eds.), ACM, 316–325.
- [44] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8.
- [45] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. arXiv:1707.06347. Retrieved from <https://arxiv.org/abs/1707.06347>
- [46] Shuanghong Shen, Qi Liu, Zhenya Huang, Yonghe Zheng, Minghao Yin, Minjuan Wang, and Enhong Chen. 2024. A survey of knowledge tracing: Models, variants, and applications. *IEEE Transactions on Learning Technologies* 17 (2024), 1858–1879.
- [47] Dongmin Shin, Yugeun Shim, Hangyeol Yu, Seewoo Lee, Byungsoo Kim, and Youngduck Choi. 2021. Saint+: Integrating temporal features for EdNet correctness prediction. In *Proceedings of the 11th International Learning Analytics and Knowledge Conference (LAK '21)*, 490–496.

- [48] Xiangyu Song, Jianxin Li, Taotao Cai, Shuiqiao Yang, Tingting Yang, and Chengfei Liu. 2022. A survey on deep learning based knowledge tracing. *Knowledge-Based Systems* 258 (2022), 110036.
- [49] Petru Soviany, Radu Tudor Ionescu, Paolo Rota, and Nicu Sebe. 2022. Curriculum learning: A survey. *International Journal of Computer Vision* 130, 6 (2022), 1526–1565.
- [50] Valentin I. Spitzkovsky, Hiyen Alshawi, and Dan Jurafsky. 2010. From baby steps to leapfrog: How “less is more” in unsupervised dependency parsing. In *Proceedings of the Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, 751–759.
- [51] Yuhao Tang, Junyu Luo, Ling Yang, Xiao Luo, Wentao Zhang, and Bin Cui. 2024. Multi-view teacher with curriculum data fusion for robust unsupervised domain adaptation. In *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2598–2611.
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, Vol. 30, 5998–6008.
- [53] Chengyi Wang, Yu Wu, Shujie Liu, Ming Zhou, and Zhenglu Yang. 2020. Curriculum pre-training for end-to-end speech translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 3728–3738.
- [54] Xin Wang, Yudong Chen, and Wenwu Zhu. 2021. A survey on curriculum learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 9 (2021), 4555–4576.
- [55] Yiwei Wang, Wei Wang, Yuxuan Liang, Yujun Cai, and Bryan Hooi. 2021. CurGraph: Curriculum learning for graph classification. In *Proceedings of the Web Conference 2021*, 1238–1248.
- [56] Ronald J. Williams and David Zipser. 1989. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation* 1, 2 (1989), 270–280.
- [57] Xiaolu Xiong, Siyuan Zhao, Eric G. Van Inwegen, and Joseph E. Beck. 2016. *Going Deeper with Deep Knowledge Tracing*. International Educational Data Mining Society.
- [58] Yu Yan, Zheng Guan, Xue Wang, Yanyou Wei, and Zhijun Yang. 2024. Knowledge tracing with soft labels via knowledge distillation and IRT-based modeling. In *Proceedings of the 2024 IEEE 3rd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA)*. IEEE, 382–386.
- [59] Sean Bin Yang, Chenjuan Guo, Jilin Hu, Bin Yang, Jian Tang, and Christian S. Jensen. 2022. Weakly-supervised temporal path representation learning with contrastive curriculum learning. In *Proceedings of the 2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2873–2885.
- [60] Yang Yang, Jian Shen, Yanru Qu, Yunfei Liu, Kerong Wang, Yaoming Zhu, Weinan Zhang, and Yong Yu. 2021. GIKT: A graph-based interaction model for knowledge tracing. In *Proceedings of the European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD '20)*. Springer, 299–315.
- [61] Chun-Kit Yeung and Dit-Yan Yeung. 2018. Addressing two problems in deep knowledge tracing via prediction-consistent regularization. In *Proceedings of the 5th Annual ACM Conference on Learning at Scale*, 1–10.
- [62] Xueqiang Zeng, Guangcheng Fu, Haoqi Chen, Qiyun Peng, and Mingwen Wang. 2025. Interactive code information integrated programming knowledge tracing. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Nicola Ferro, Maria Maistro, Gabriella Pasi, Omar Alonso, Andrew Trotman, and Suzan Verberne (Eds.), ACM, 2728–2732.
- [63] Jiani Zhang, Xingjian Shi, Irwin King, and Dit-Yan Yeung. 2017. Dynamic key-value memory networks for knowledge tracing. In *Proceedings of the 26th International Conference on World Wide Web*, 765–774.
- [64] Jing Zhao, Jiajie Xu, Yuan Xu, Junhua Fang, Pingfu Chao, and Xiaofang Zhou. 2024. CCML: Curriculum and contrastive learning enhanced meta-learner for personalized spatial trajectory prediction. *IEEE Transactions on Knowledge and Data Engineering* 36 (2024), 4499–4514.
- [65] Mingjun Zhao, Haijiang Wu, Di Niu, and Xiaoli Wang. 2020. Reinforced curriculum learning on pre-trained neural machine translation models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34, 9652–9659.
- [66] Yiyun Zhou, Zheqi Lv, Shengyu Zhang, and Jingyuan Chen. 2025. Disentangled knowledge tracing for alleviating cognitive bias. In *Proceedings of the ACM on Web Conference 2025 (WWW '25)*. Guodong Long, Michale Blumstein, Yi Chang, Liane Lewin-Eytan, Zi Helen Huang, and Elad Yom-Tov (Eds.), ACM, 2633–2645.

Received 11 August 2025; revised 4 April 2026; accepted 13 May 2026